Transformers

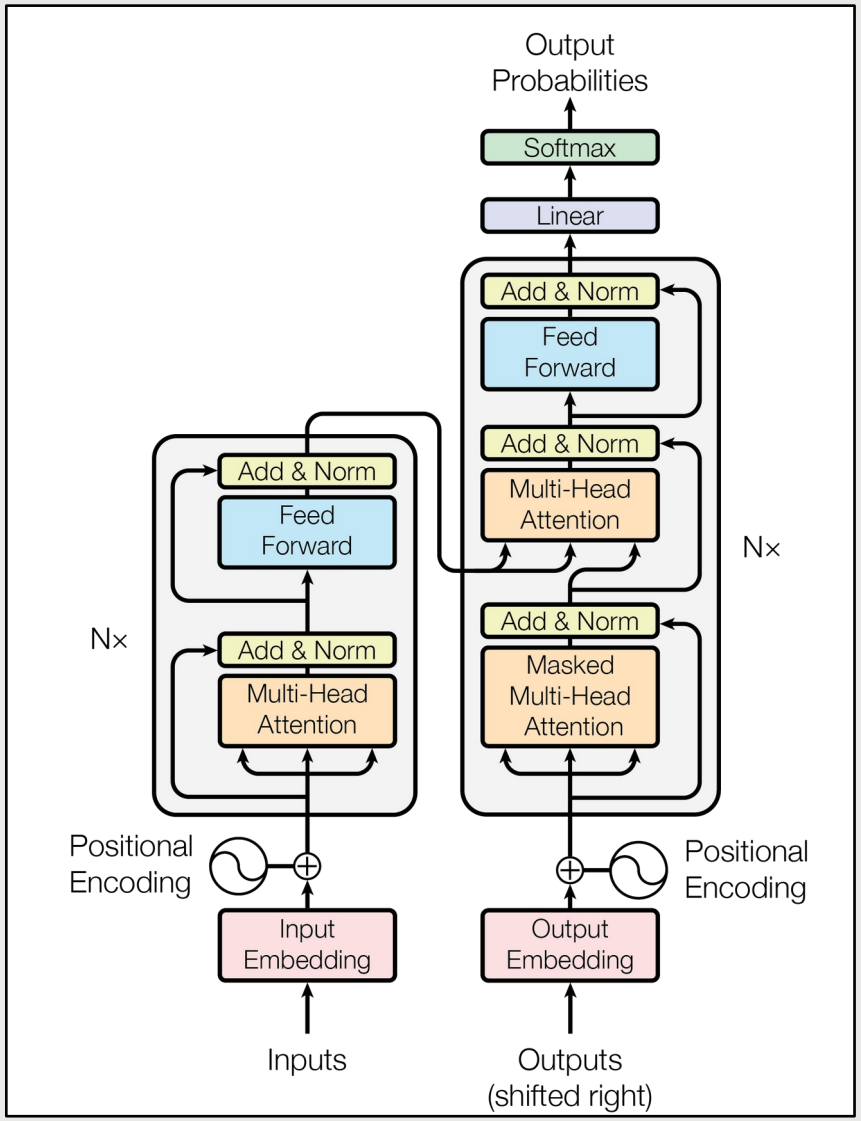
Part II

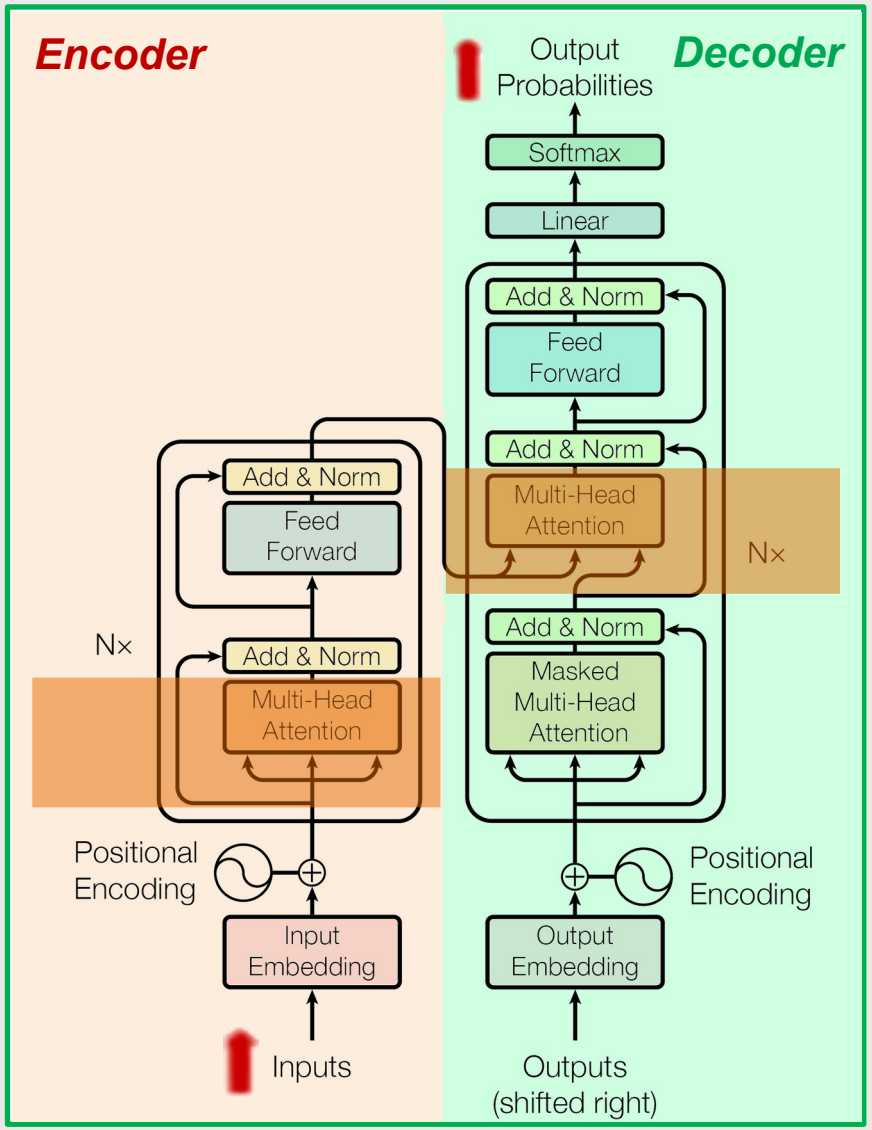
Aug. 2020

Review of Part I

Self Attention Mechanism

Encoder – Decoder Transformer





Transformers – Part II

Parallelization Implementation Issues

Positional Information

Building Transformer Block

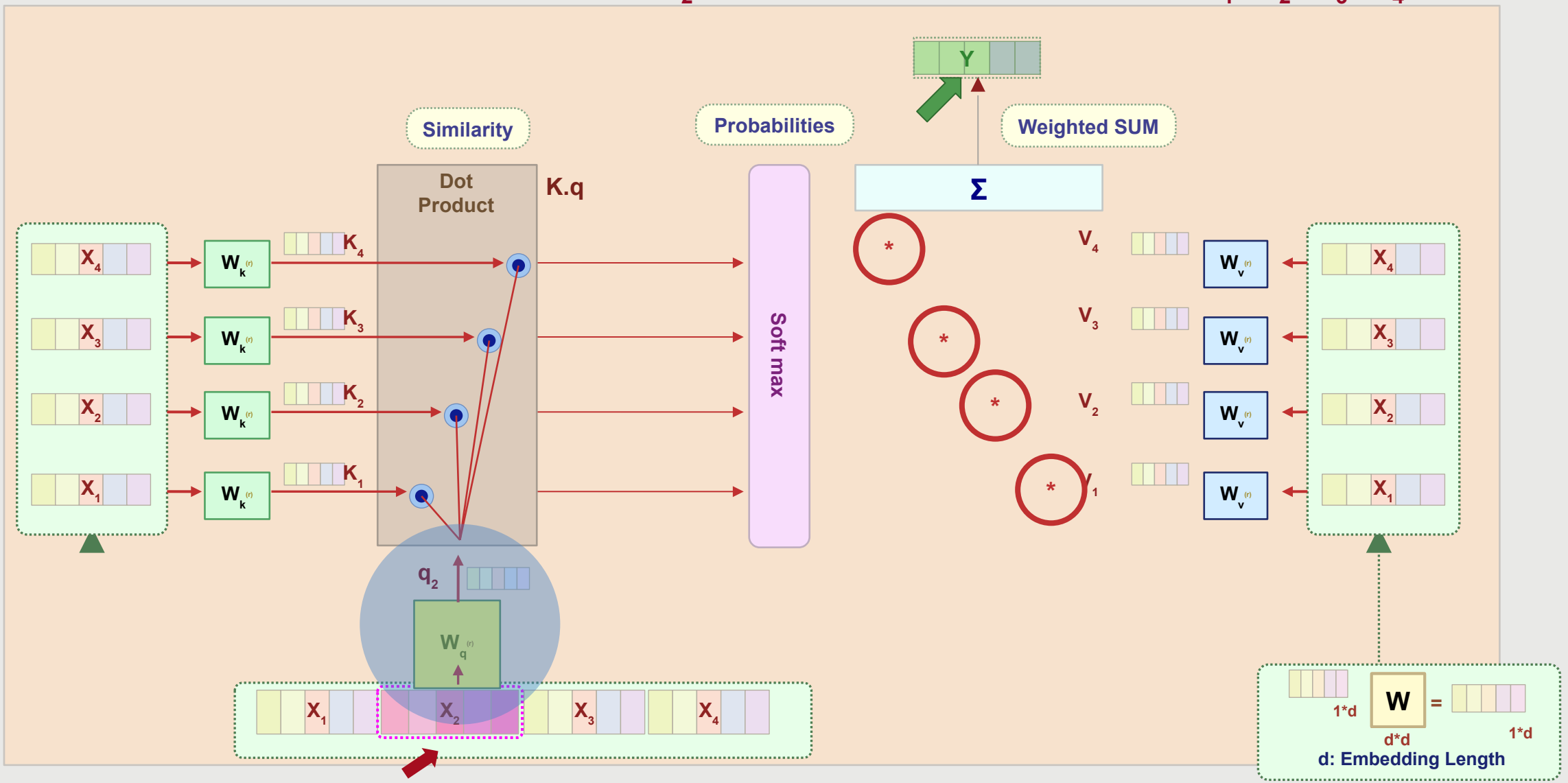
Transformer Decoder

Transformer Encoder as a Classifier

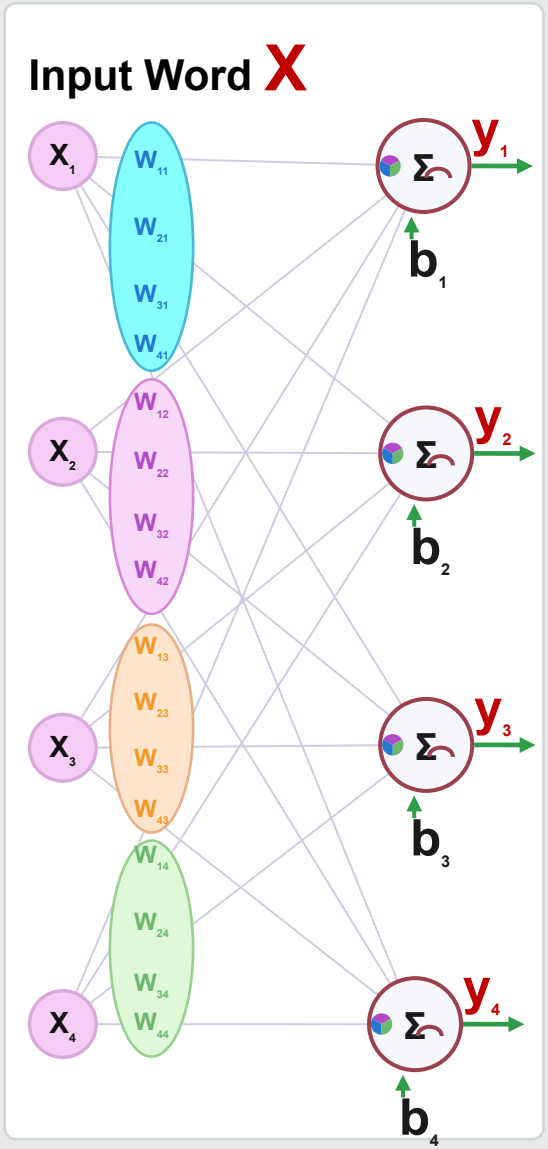
What is Next ?

Parallelization Implementation Issues

Self Attention Mechanism (Linear Transformation of Inputs)
(Self Attention of word X_2 w.r.t All words in Sentence (X_1, X_2, X_3, X_4)



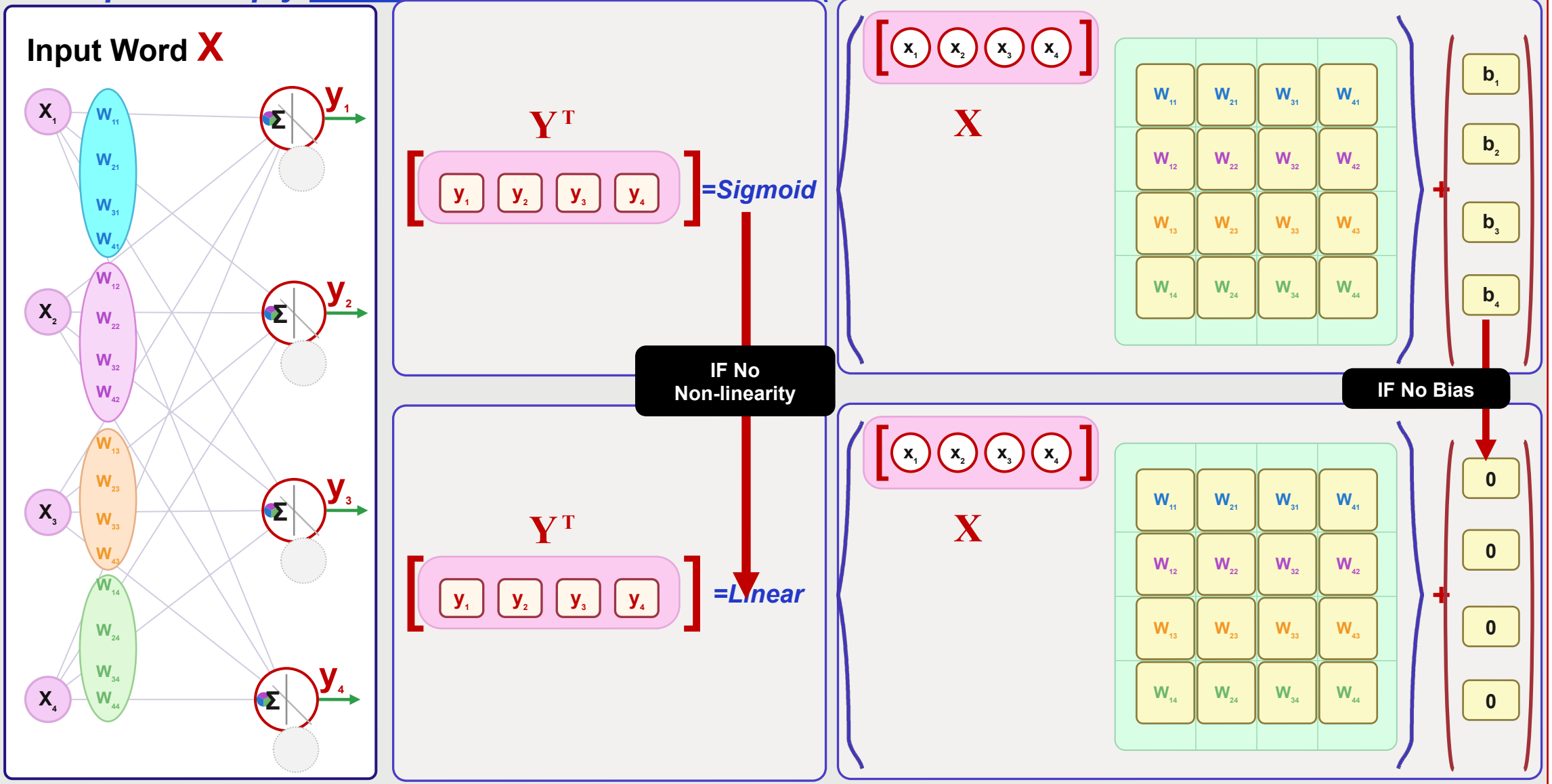
Using FF NN for Multiplication of a Vector by a Matrix

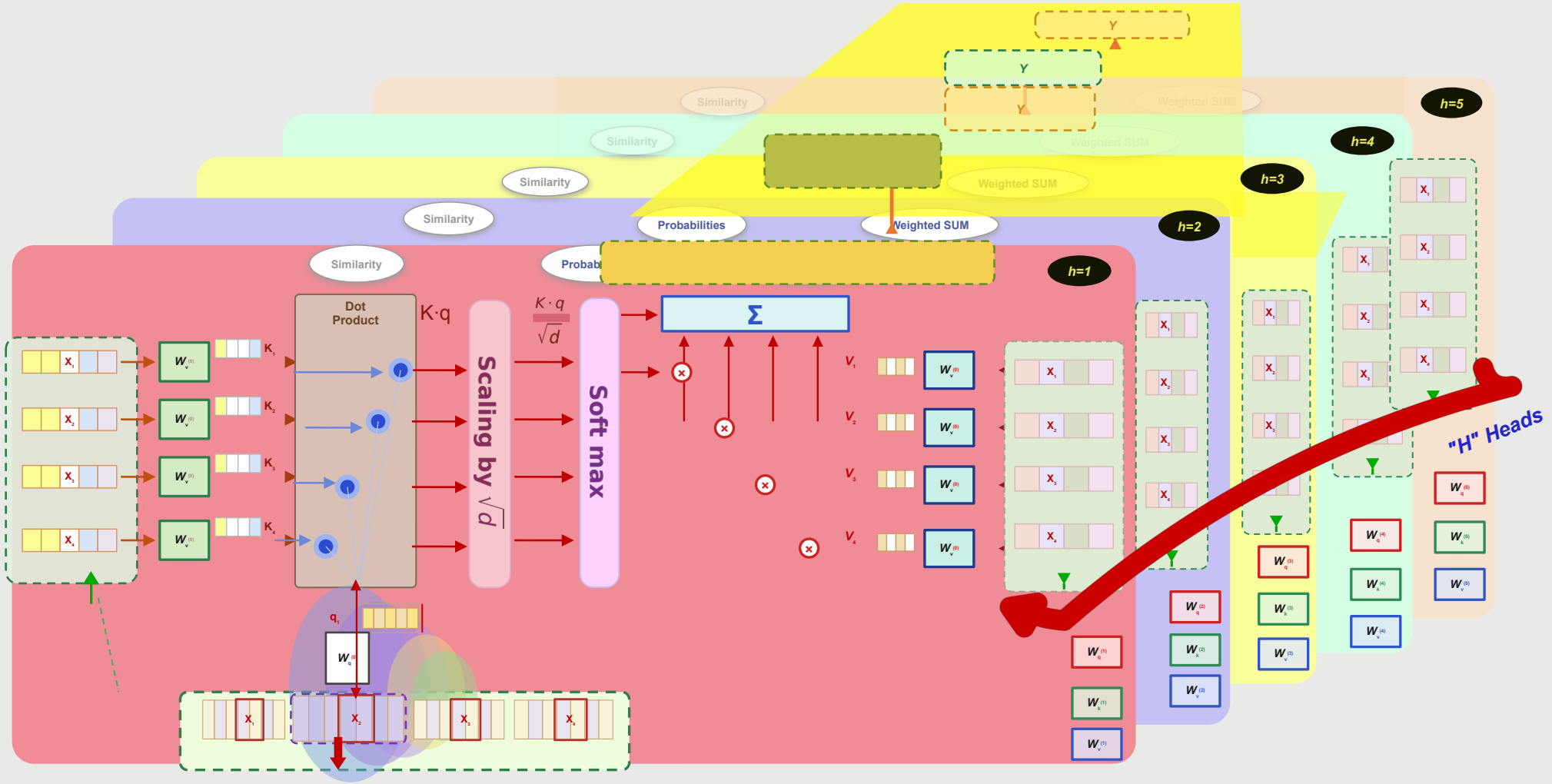


$$\begin{bmatrix} y_1 & y_2 & y_3 & y_4 \end{bmatrix}^T = Sigmoid\left(\begin{bmatrix} x_1 & x_2 & x_3 & x_4 \end{bmatrix} X \begin{bmatrix} w_{11} & w_{21} & w_{31} & w_{41} \\ w_{12} & w_{22} & w_{32} & w_{42} \\ w_{13} & w_{23} & w_{33} & w_{43} \\ w_{14} & w_{24} & w_{34} & w_{44} \end{bmatrix} + \begin{pmatrix} b_1 \\ b_2 \\ b_3 \\ b_4 \end{pmatrix} \right)$$

Using FF NN for Multiplication of ONE Vector by a Matrix

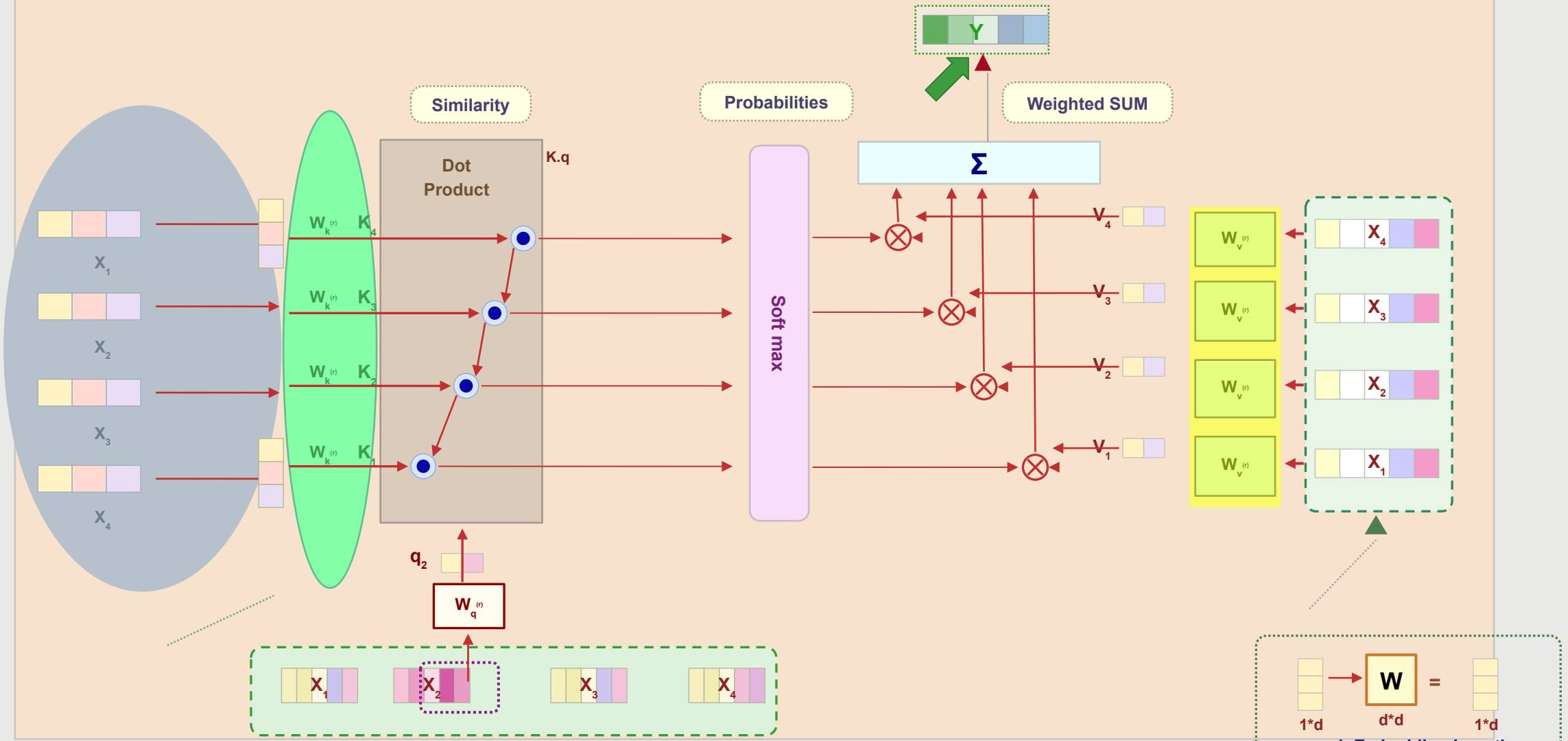
Example: Multiply Vector of Current Word X by W_Q (Query) {Remember: W_Q is Squared Matrix}

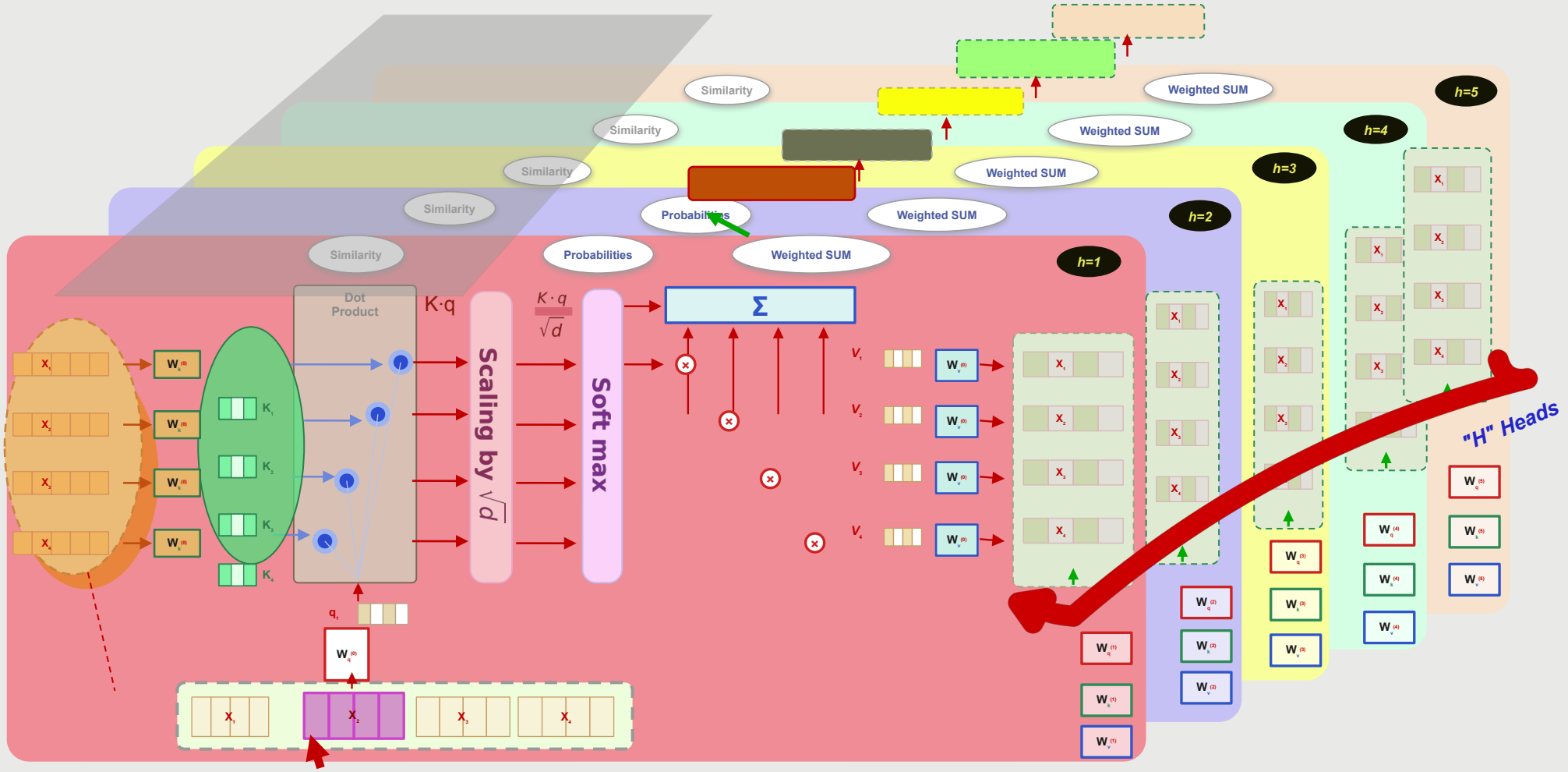




Multi Head Self Attention of Word X_2

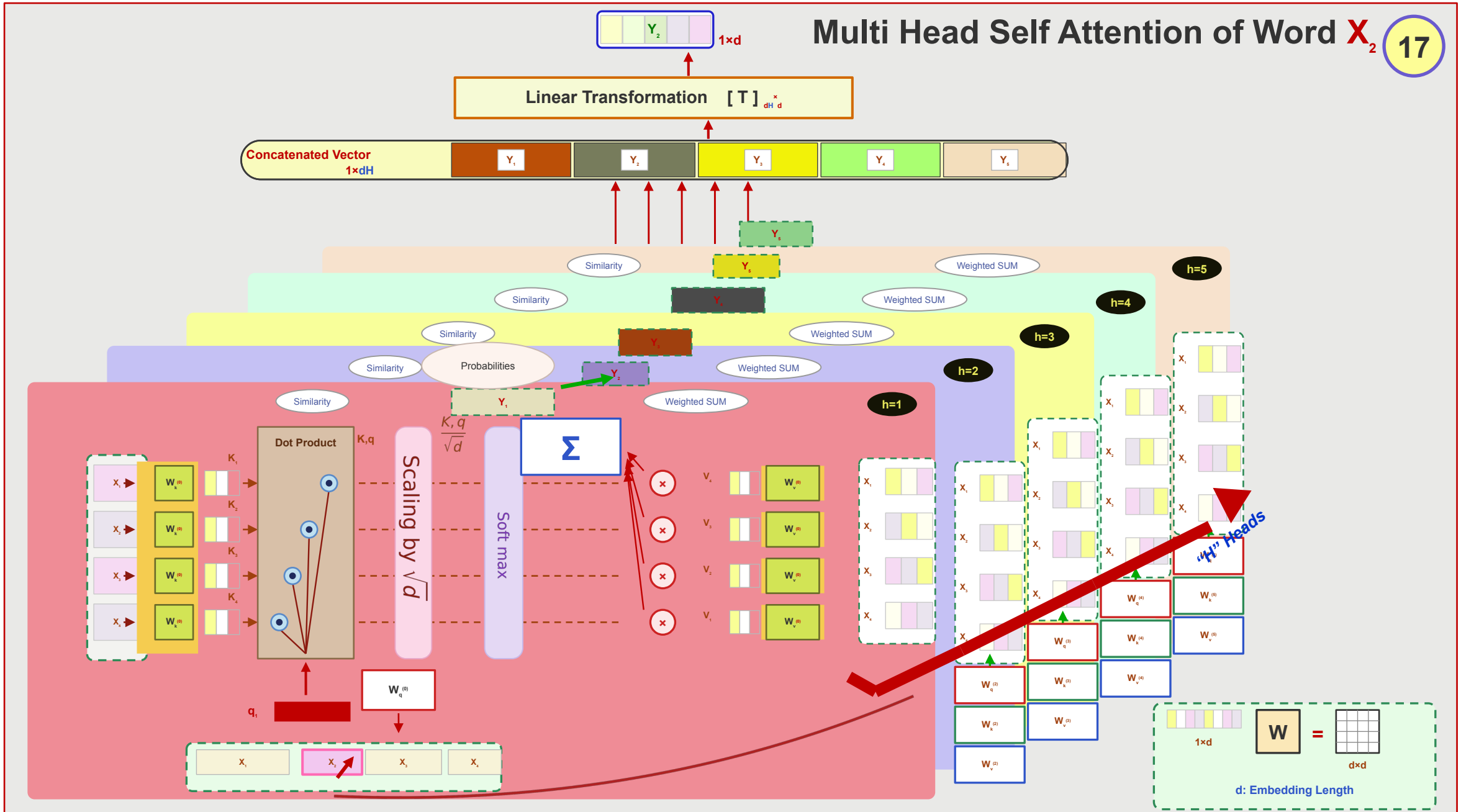
Self Attention Mechanism (Linear Transformation of Inputs)
(Self Attention of word X_2 w.r.t All words in Sentence (X_1, X_2, X_3, X_4)





Multi Head Self Attention of Word X_2

Multi Head Self Attention of Word X_2

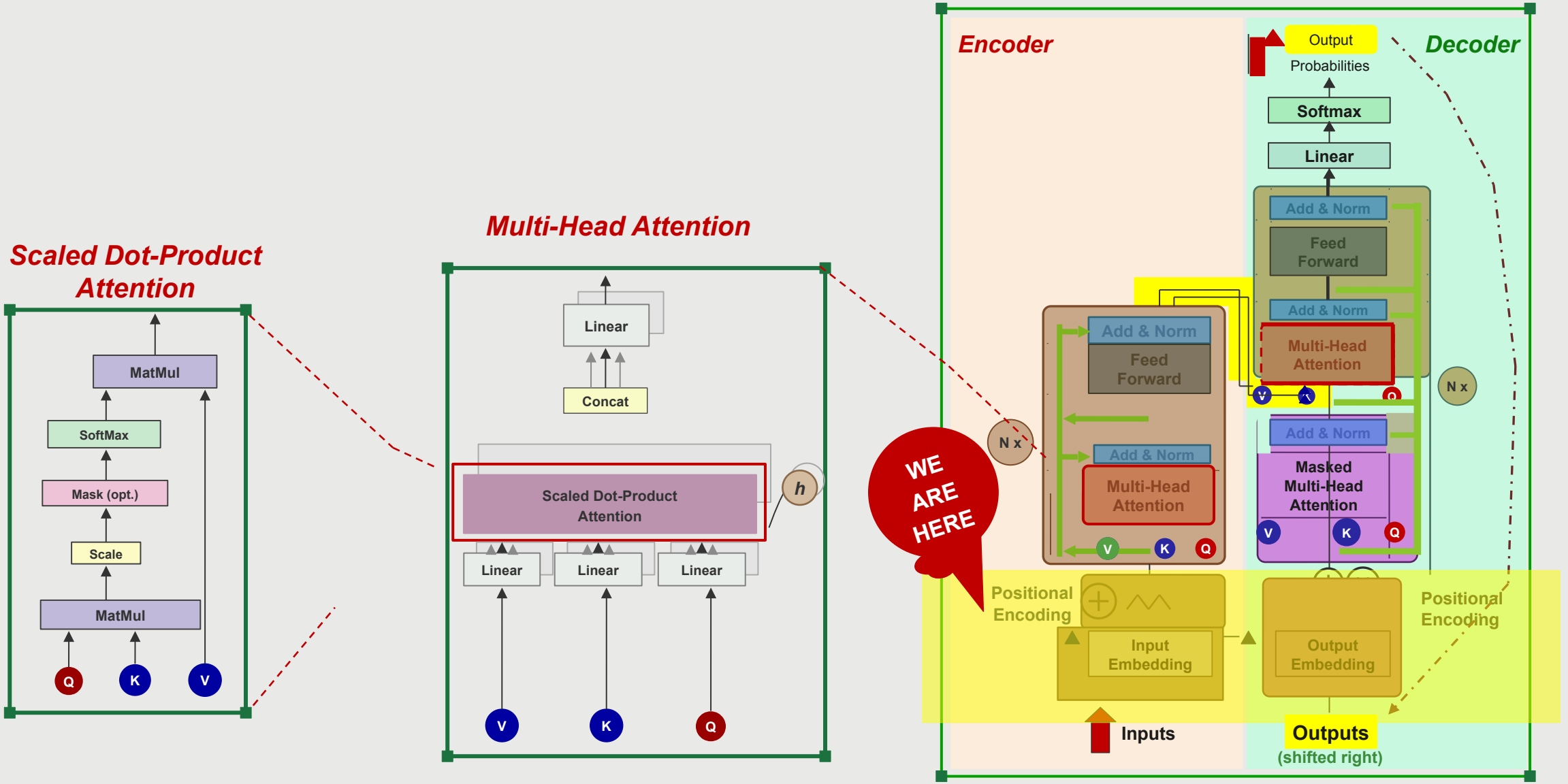


Adding Position Information

[1] Position Encoding

[2] Position Embedding

Encoder – Decoder Transformer



Pre-processing of Input Sequence

Sequence of words with Length N

Word 0 | Word 1 | Word 2 | Word 3 | Word 4 | Word 5 | ... | Word $_{N-2}$ | Word $_{N-1}$

Word Embedding

e.g. word2vec

X_0 X_1 X_2 X_3 X_4 X_5 ... X_{N-2} X_{N-1}

Position Information

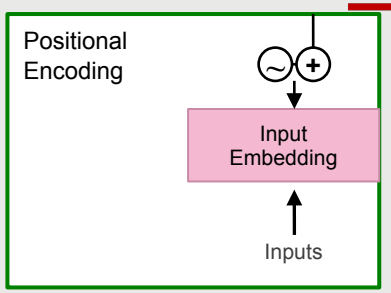
Pos 0 | Pos 1 | Pos 2 | Pos 3 | Pos 4 | Pos 5 | ... | Pos $_{N-2}$ | Pos $_{N-1}$

$+\delta$ $+\delta$ $+\delta$ $+\delta$ $+\delta$ $+\delta$

Final Embedding

Combine Word Encoding and Position Information (for Each Word)

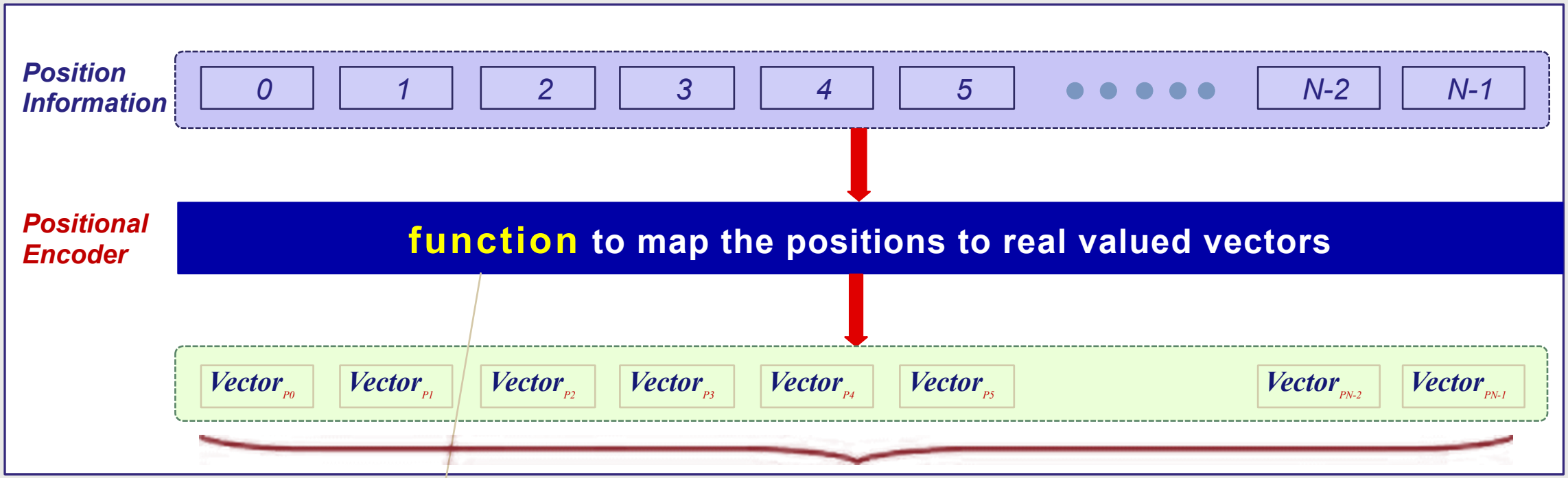
I/P 0 | I/P 1 | I/P 2 | I/P 3 | I/P 4 | I/P 5 | ... | I/P $_{N-2}$ | I/P $_{N-1}$



Input to Encoder

Input to first Self Attention Layer in Transformer

[1] Position Encoding



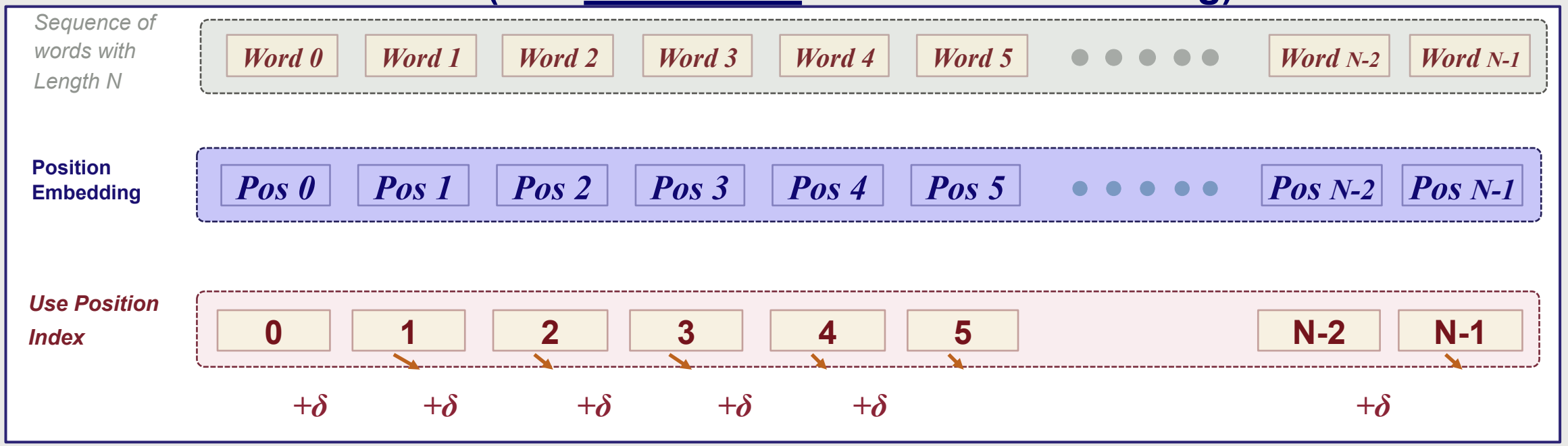
[1] Function with **Fixed** Parameters

[2] Function with **TRAINABLE** Parameters

Real valued vectors representing positions (Vector for each position)

Vector Length = Word Embedding length

Suggested Positional Encoders
(Use Word Index as Position Encoding)

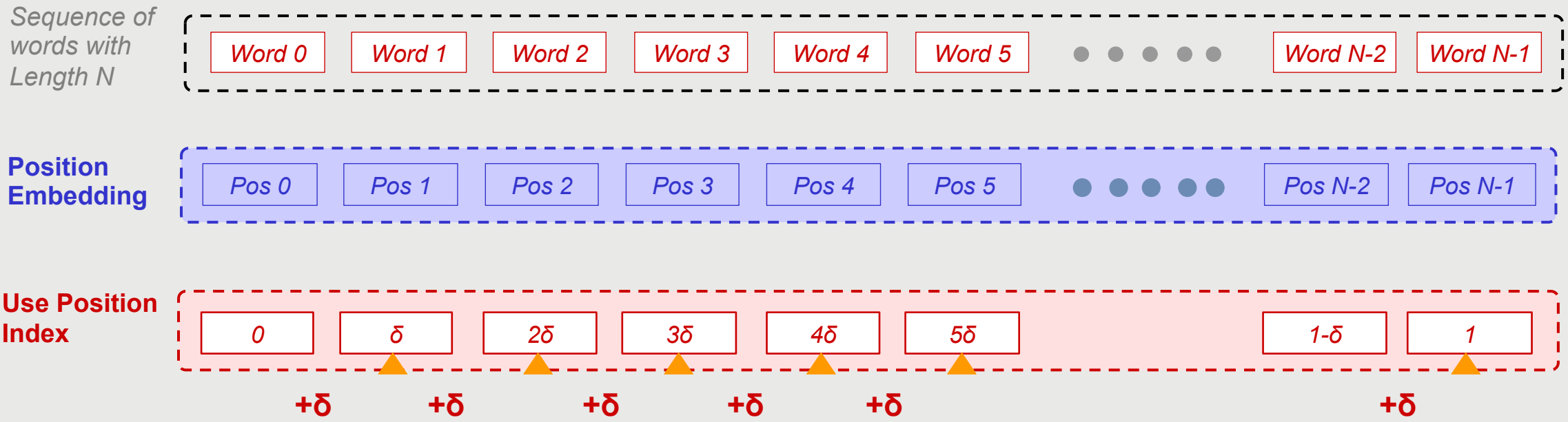


Use $\delta = 1$ First position Encoding **=0** , Last Position Encoding = **N-1** (Depends on Seq. Length)

Problems:

- **If N is large** (e.g. seq. Length = 1024) **Large position Values will dominate** when Combine Position Embedding to Word Embedding
- If System is trained with Max seq length =256 **how can the system deal with Larger sequence values**
- Position Encoding is **scalar** and Word embedding is **vector** (how to Combine them ?)

Suggested Positional Encoders
(Normalize Seq. Length to "1")

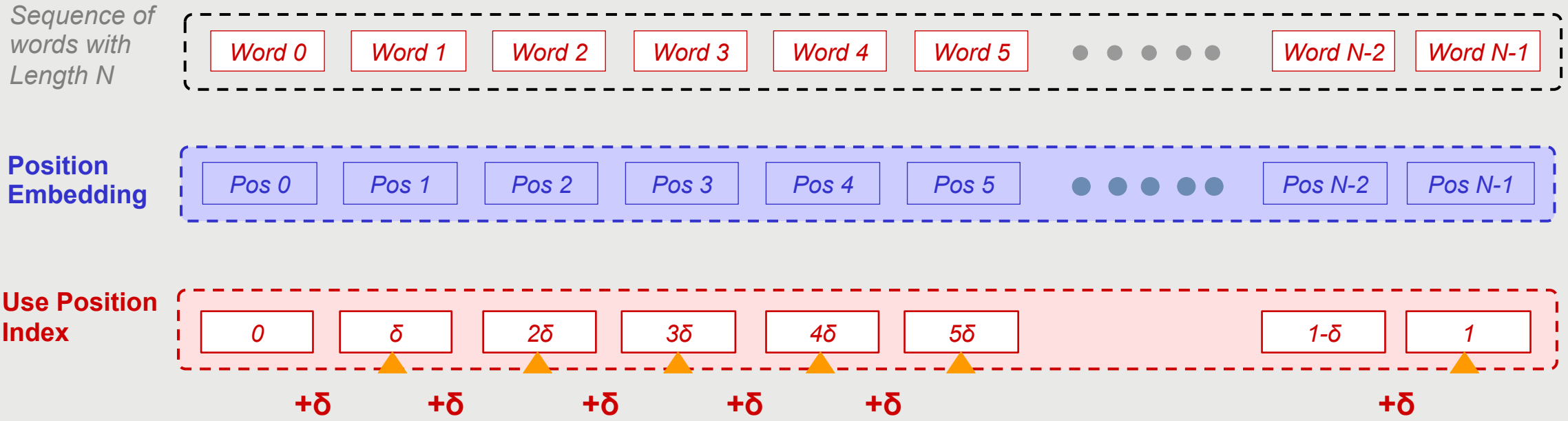


*Use **δ=1/Seq. Length** First position Encoding=**0**, Last Position Encoding= **1** **(Independent of Seq. Length)***

Problems:

- ***δ** Value depends on **Seq. Length** (δ is small for **long** seq and large for **short** sequence)*
- *In other words, Encoding value of position # 4 in **short** seq will **differ** from position #4 in **long** seq **(though they are same position)***
- *Position Encoding is **scalar** and Word embedding is **vector** (how to Combine them ?)*

Suggested Positional Encoders
(Rules Controlling Good Positional Encoder)

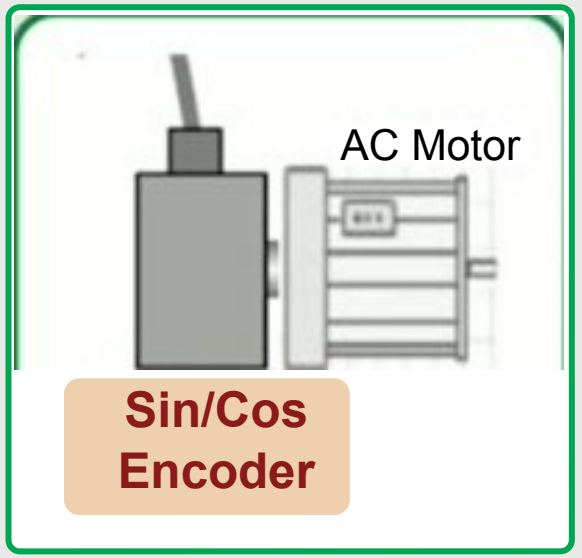


[1] ***δ value should be same*** *for Long sequence and short sequence*
(same positions will be encoded with same values irrespective of the length of the sequence)

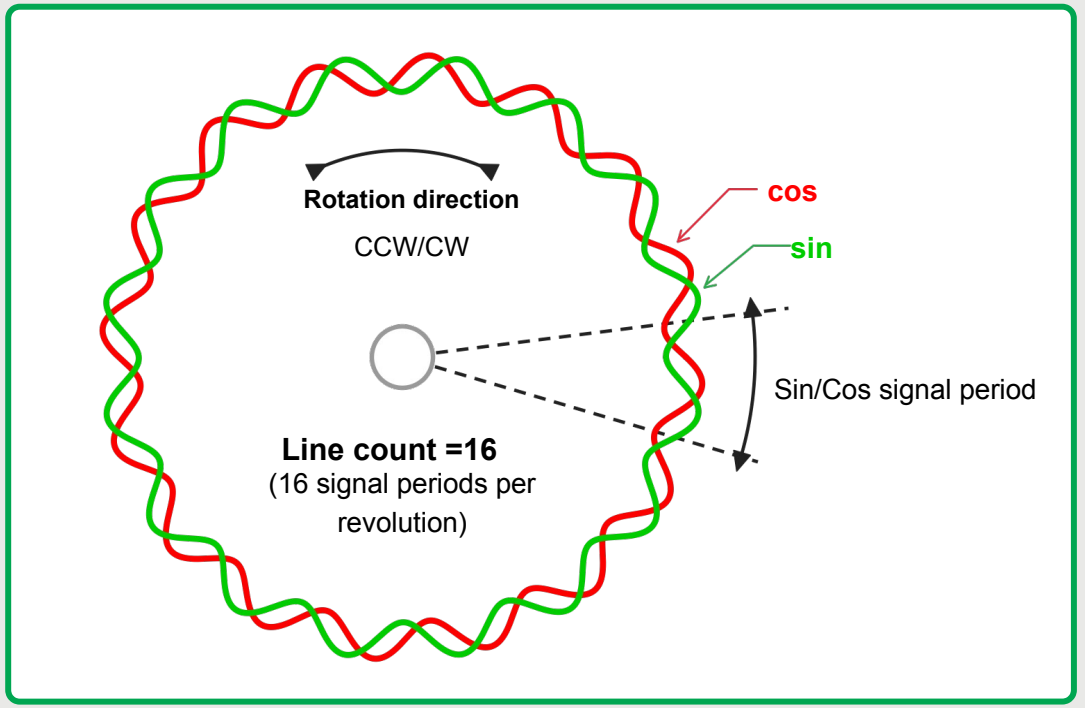
[2] Range of position encoded values should NOT depend on sequence length
(predefined fixed range e.g. from 0 to 1 , form -1 to 1 , ...)

[3] Position Encoding should be a VECTOR *(not scalar) and of* ***same dimension as Word Embedding***
(in order to easily combine Position Encoding and Word Encoding vectors)

Sine/Cosine Encoders
(an industrial application of Motor Control System)



Sin/Cos Encoder Output Signals

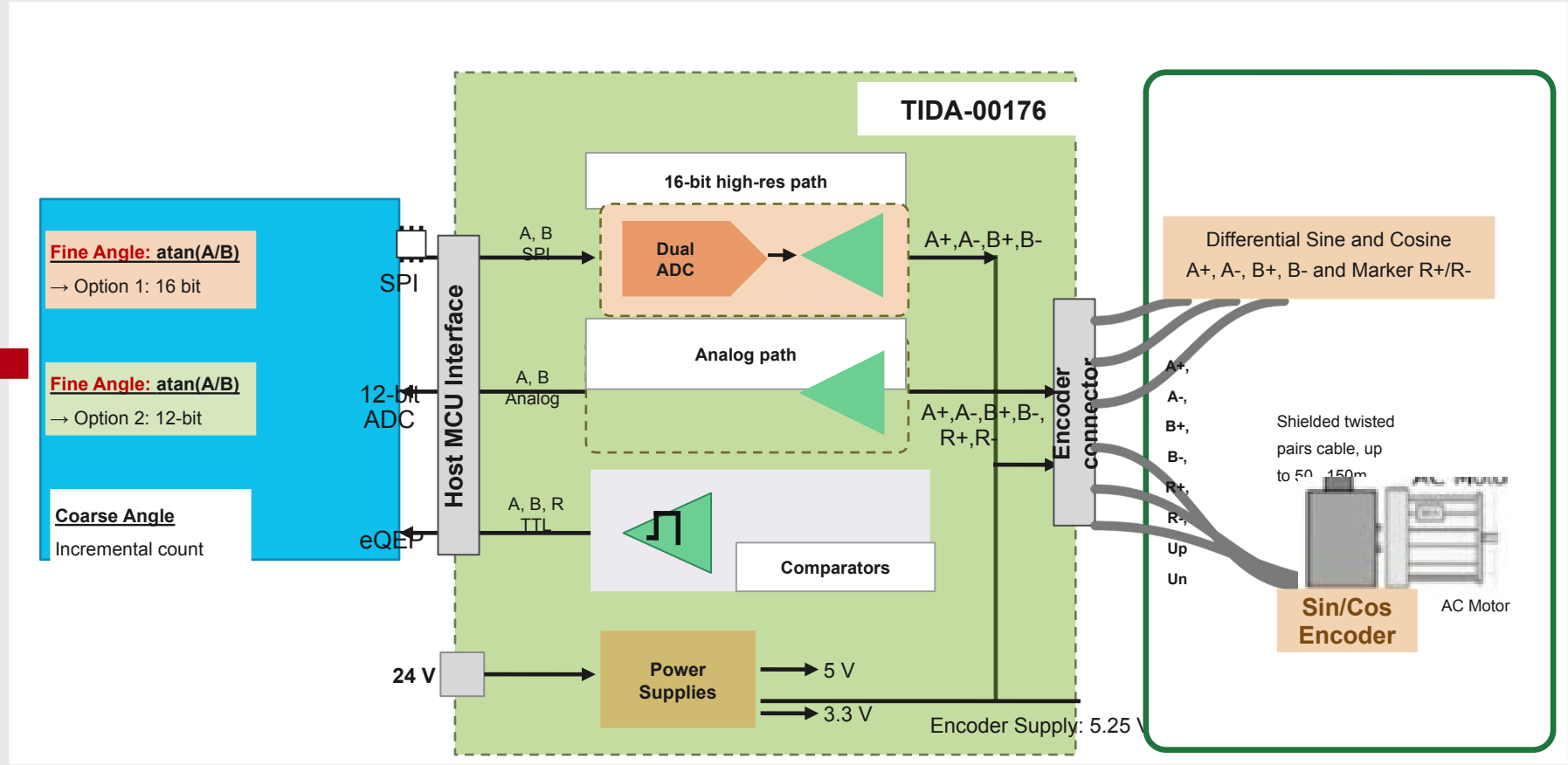


Incremental rotary position encoders are used in many applications to measure angular position and speed.

For More Info:

EXTRACTION OF HIGH RESOLUTION POSITION INFORMATION FROM SINUSOIDAL ENCODERS
J. Burke, J. F. Moynhan, K. Unterkofler

Sin/Cos Encoders
(an industrial application of Motor Control System)

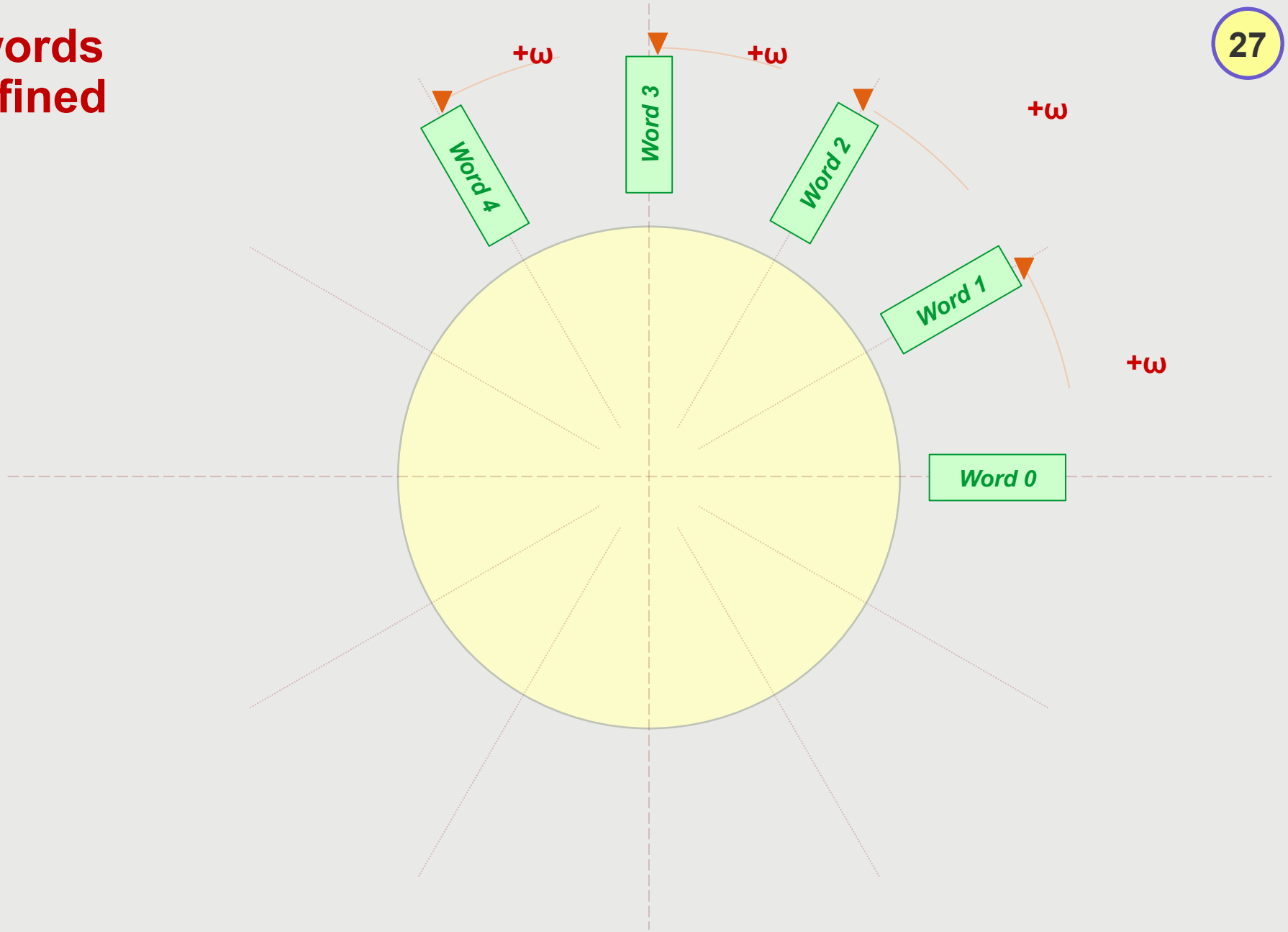


EMC-compliant interface to Sin/Cos incremental position encoder switch

1-VPP Differential analog output signals

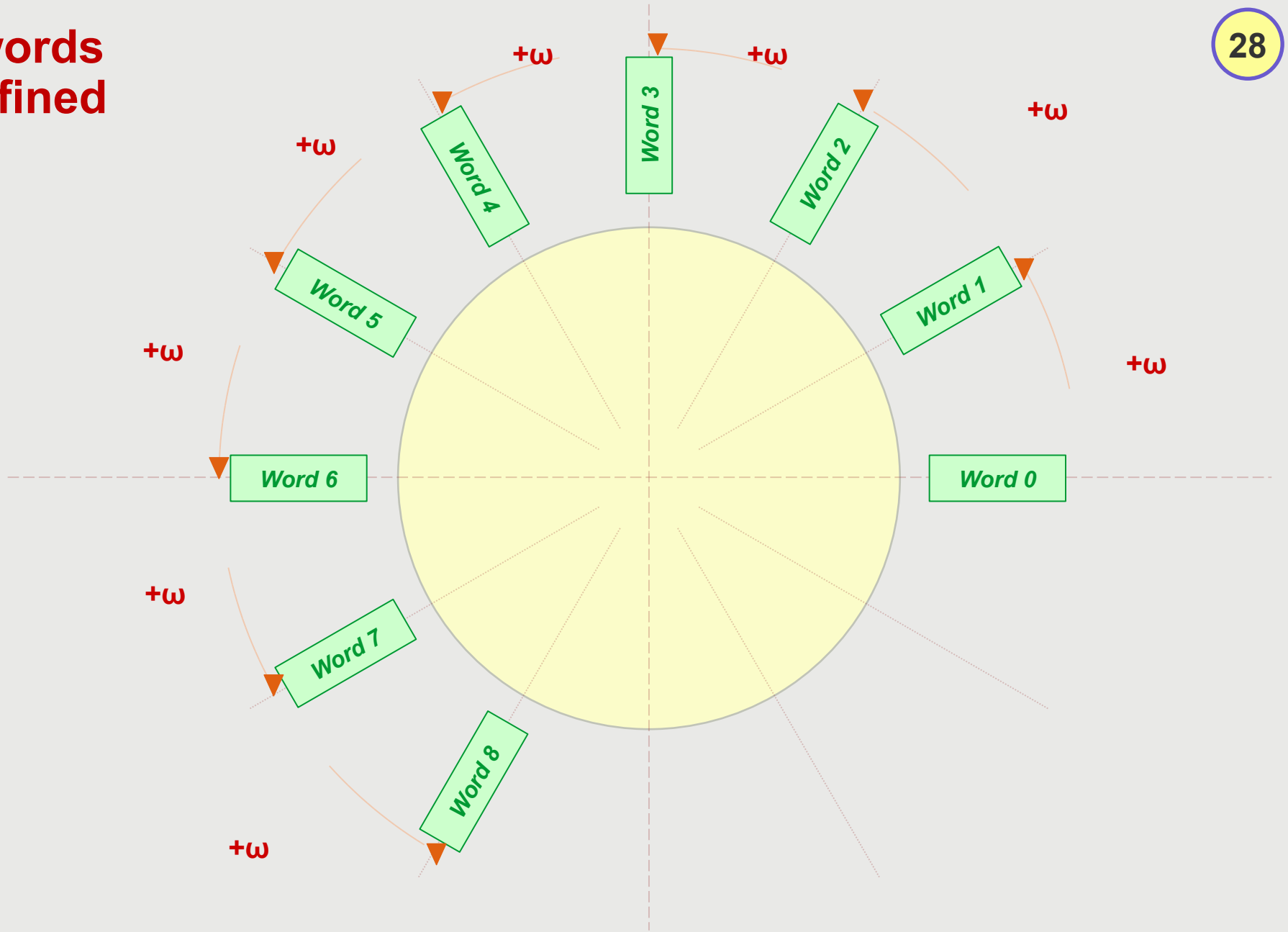
Sequence of 5 words Positions are defined as angles

Sin(ω t)
Cos(ω t)
t=0,1,2,3,....



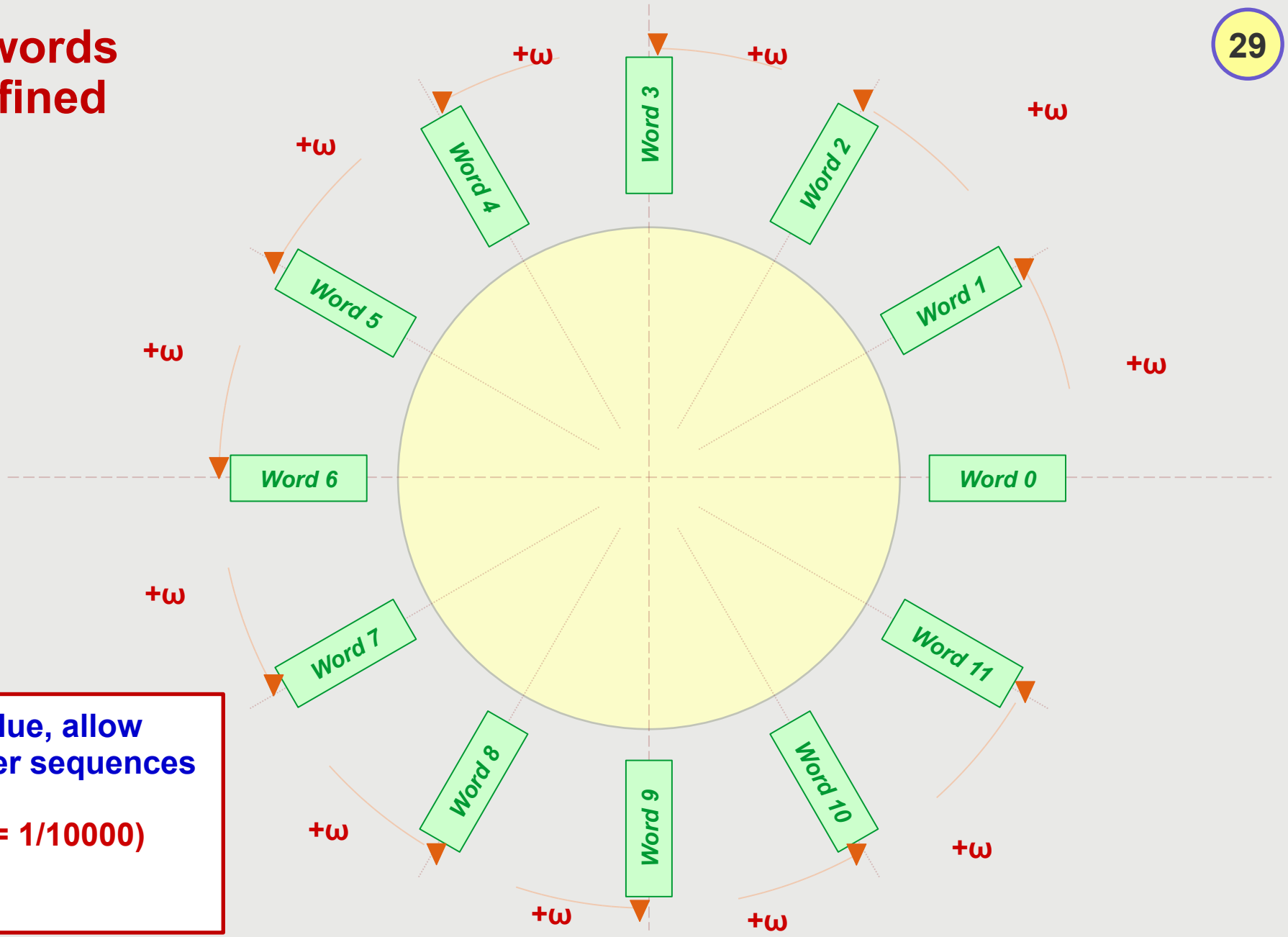
Sequence of 9 words Positions are defined as angles

Sin(ω t)
Cos(ω t)
t=0,1,2,3,....



Sequence of 12 words Positions are defined as angles

Sin(ω t)
Cos(ω t)
t=0,1,2,3,....



Using smaller ω value, allow representation of longer sequences

(practically use ω = 1/10000)

Vector of Pos "t"

$$\begin{bmatrix} \sin(\omega t) \\ \cos(\omega t) \end{bmatrix}$$

Vector of Pos "t+δ"

$$\begin{bmatrix} \text{Sin}\,(\omega(t+\delta)) \\ \cos(\omega(t+\delta)) \end{bmatrix}$$

Remember:

$$\sin(a+b) = \sin(a)\cos(b) + \cos(a)\sin(b)$$
$$\cos(a+b) = \cos(a)\cos(b) - \sin(a)\sin(b)$$

$$\sin(\omega(t+\delta)) = \sin(\omega t + \omega\delta) = \sin(\omega t)\cos(\omega\delta) + \cos(\omega t)\sin(\omega\delta)$$

$$\cos(\omega(t+\delta)) = \cos(\omega t + \omega\delta) = \cos(\omega t)\cos(\omega\delta) - \sin(\omega t)\sin(\omega\delta)$$

$$\begin{bmatrix} \text{Sin}\,(\omega(t+\delta)) \\ \cos(\omega(t+\delta)) \end{bmatrix} = \begin{bmatrix} \cos(\omega\delta) & \sin(\omega\delta) \\ -\sin(\omega\delta) & \cos(\omega\delta) \end{bmatrix} \begin{bmatrix} \sin(\omega t) \\ \cos(\omega t) \end{bmatrix}$$

Vector of Pos "t+δ"

Vector of Pos "t"

Independent of Position "t"

From Computer Graphics:

Rotation Matrix of Vector at Pos "t" with angle "ωδ"

e.g "ω", "2ω", "3ω", ...

Vector at Pos "t"

$$\begin{bmatrix} \sin(\omega t) \\ \cos(\omega t) \end{bmatrix} \rightarrow \begin{bmatrix} \sin(a) \\ \cos(a) \end{bmatrix}$$

Vector at Pos "t+δ"

$$\begin{bmatrix} \text{Sin}\,(\omega(t+\delta)) \\ \cos\,(\omega(t+\delta)) \end{bmatrix} = \begin{bmatrix} \sin(\omega t)\cos(\omega\delta) + \cos(\omega t)\sin(\omega\delta) \\ \cos(\omega t)\cos(\omega\delta) - \sin(\omega t)\sin(\omega\delta) \end{bmatrix} \rightarrow \begin{bmatrix} \sin(a)\cos(b) + \cos(a)\sin(b) \\ \cos(a)\cos(b) - \sin(a)\sin(b) \end{bmatrix}$$

Let $\omega t = a$, $\omega\delta = b$

$\sin^2(a) + \cos^2(a) = 1$

Squared Distance Between Vector of Pos "t+δ" and Vector of Pos "t" [Two Consecutive Positions]

$$[\sin(a)\cos(b) + \cos(a)\sin(b) - \sin(a)]^2 + [-\sin(a)\sin(b) + \cos(a)\cos(b) - \cos(a)]^2 =$$

$$[\sin(a)(\cos(b)-1) + \cos(a)\sin(b)]^2 + [-\sin(a)\sin(b) + \cos(a)(\cos(b)-1)]^2 =$$

$$[\sin(a)^2(\cos(b)-1)^2 + \cos^2(a)\sin^2(b) + 2\sin(a)\cos(a)\sin(b)(\cos(b)-1)] +$$
$$[\sin^2(a)\sin^2(b) + \cos^2(a)(\cos(b)-1)^2 - 2\sin(a)\cos(a)\sin(b)(\cos(b)-1)] =$$

$$\sin(a)^2(\cos(b)-1)^2 + \cos^2(a)\sin^2(b) + \sin^2(a)\sin^2(b) + \cos^2(a)(\cos(b)-1)^2 =$$

$$[\sin(a)^2 + \cos^2(a)](\cos(b)-1)^2 + [\sin(a)^2 + \cos^2(a)](\sin^2(b)) = (\cos(b)-1)^2 + \sin^2(b) =$$

$$\cos^2(b) - 2\cos(b) + 1 + \sin^2(b) = 2 - 2\cos(b) = 2(1-\cos(b))$$

$$= \mathbf{2(1 - \cos(\omega\delta))}$$

Irrespective of Position "t"

Sin/Cos Positional Encoder

Vector of Dimension "2" to define Position

$$\begin{bmatrix} \sin(\omega t) \\ \cos(\omega t) \end{bmatrix}$$

Where "t" is the step (position) and ω is the frequency of trigonometric function Sine/Cosine

This vector can be used to define word position at step "t" , t=0,1,2,3,4,5,6,7,....

- **Values are between -1 and 1** ***(bounded values)***

  Distance between vectors representing positions "t" and "t+1" is constant irrespective of "t" [Distance (Pos1, Pos2) = Distance (Pos90, Pos91)]

- ***"t" may take any positive value*** ***(No limitation of length of Sequence)***

- **Position Vector Length = 2 (Cannot be combined with Word Embedding Vector) ??**

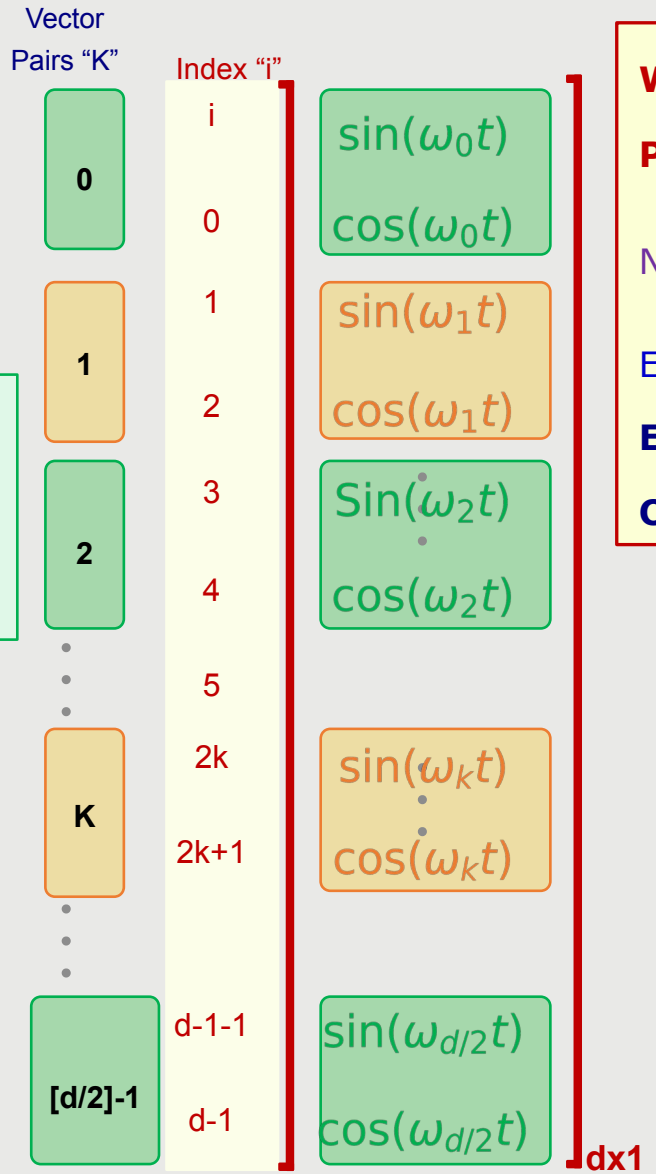
Sin/Cos Positional Encoder

Positional Vector with Same dimension as Embedding dimension

$$\begin{bmatrix} \sin(\omega t) \\ \cos(\omega t) \end{bmatrix}$$

Sin/Cos pairs
Vector Represents
Position "t"

Any "ω" is accepted



Word Embedding Dimension **"d"** ***[Even Num]***

Position Encoding Dimension **"d"**

Number of Vector Pairs "**K"=d/2** [0, 1, 2,... d/2-1]

Elements Index **"i"** [0, 1, 2, ..., d-1]

Even Index **"i"** is **Sine Term** , **i=2*k**

Odd Index **"i"** is **Cosine Term, i=2*k+1**

Example:

d=100

K=[0 ..49]

i=[0 .. 99]

Sin [0,2,4,...]

Cos [1,3,5,...]

Use different "ω" values for different "Pairs"
Select "ω" Value Based on "Pair" index k

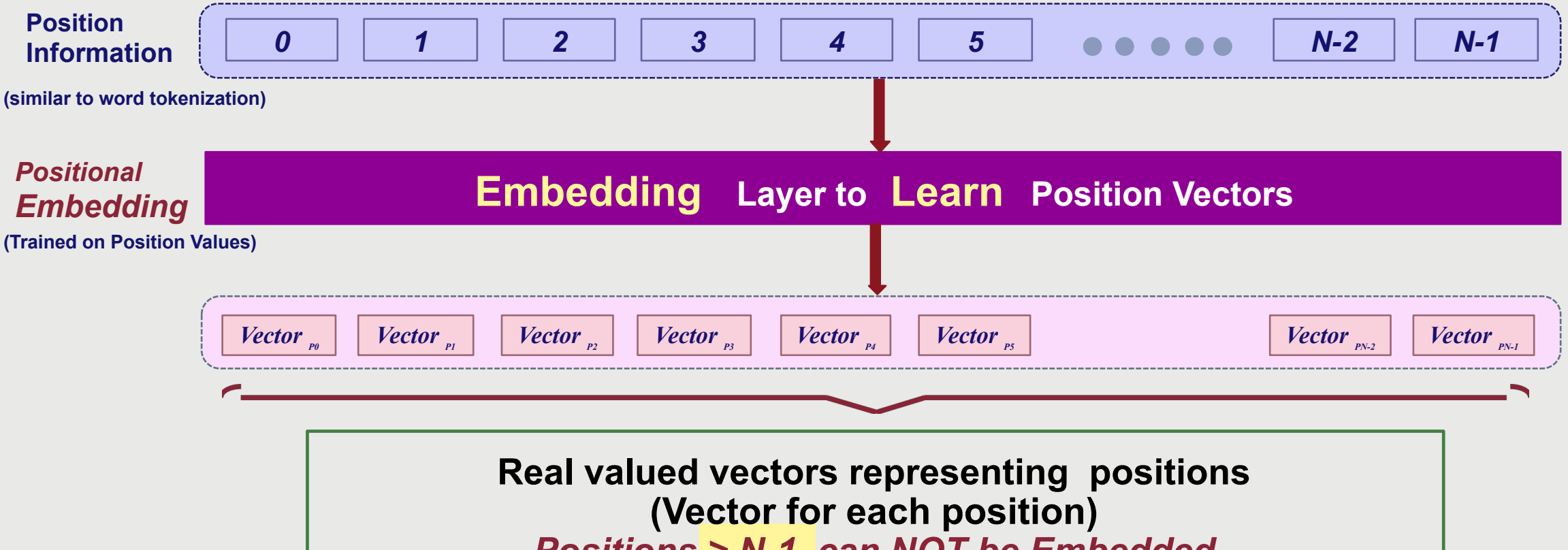
$$\omega_K = \frac{1}{10{,}000^{2k/d}}$$

Largest "ω" value = 1 (Used in First Pair)
Smallest "ω" value ≈ 1/10000 (Used in Last Pair)

Note:
"ω" Value depends on Embedding Dimension "d" **ONLY**
"ω" Value does **NOT** depend on Sequence Length

[2] Position Embedding

(Similar to Word Embedding)



Real valued vectors representing positions
(Vector for each position)

Positions > N-1 can NOT be Embedded

(Must Train Embedding on Max Expected Seq. Length)

Vector Length = Word Embedding length

Building Transformer Block

Adding

- Residual Connections
- Layer Normalization
- Feed Forward Network

Multi Head Self Attention Block

Input:

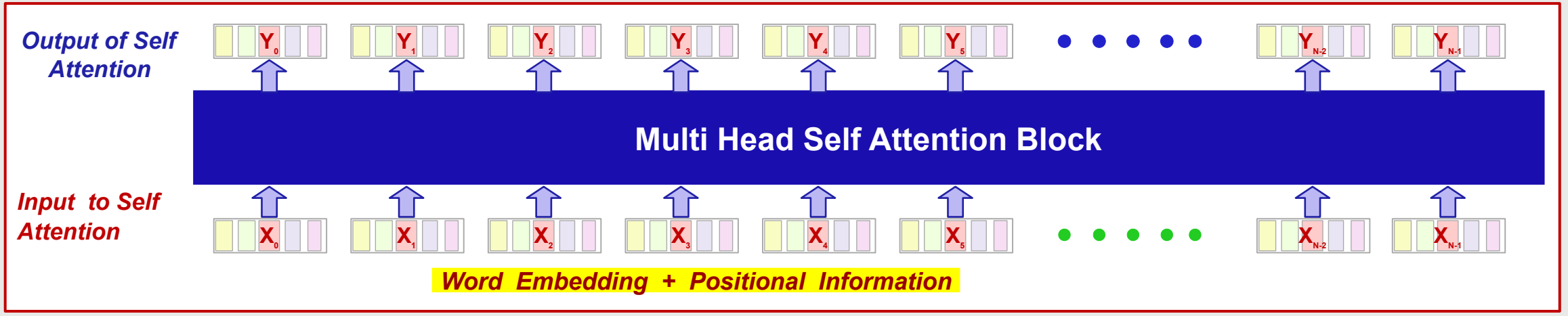
Word Embedding + Positional Information [First Transformer Block]
Output of Previous Transformer Block [Stacked Transformer Blocks]

Output:

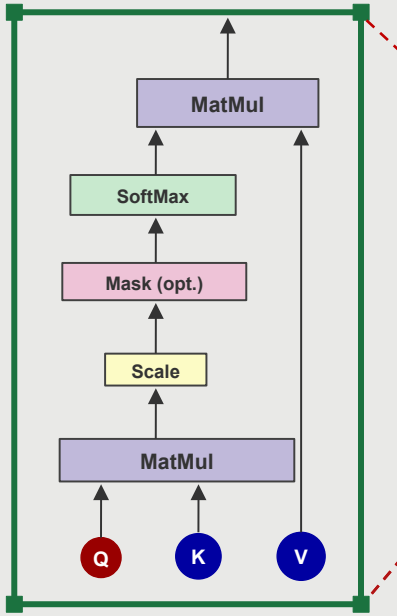
Encoded Vectors of Input Sequence [With Same Input Vectors Dimension]

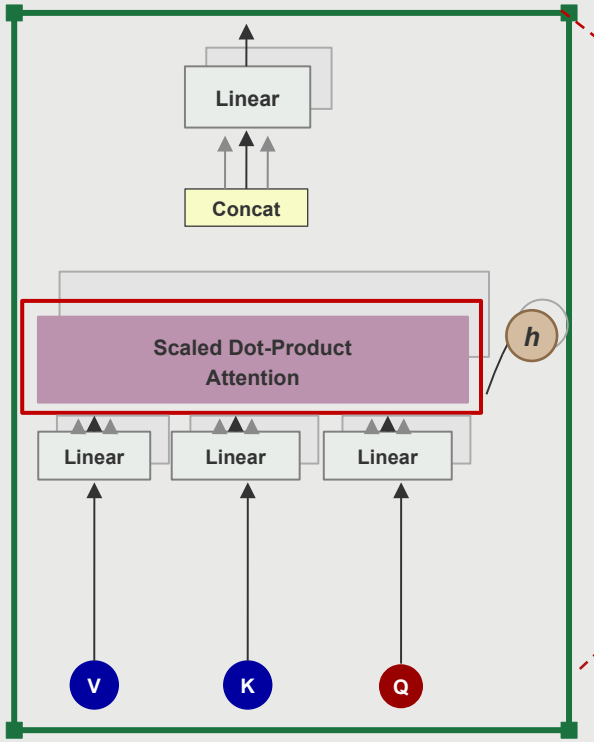
Remember:

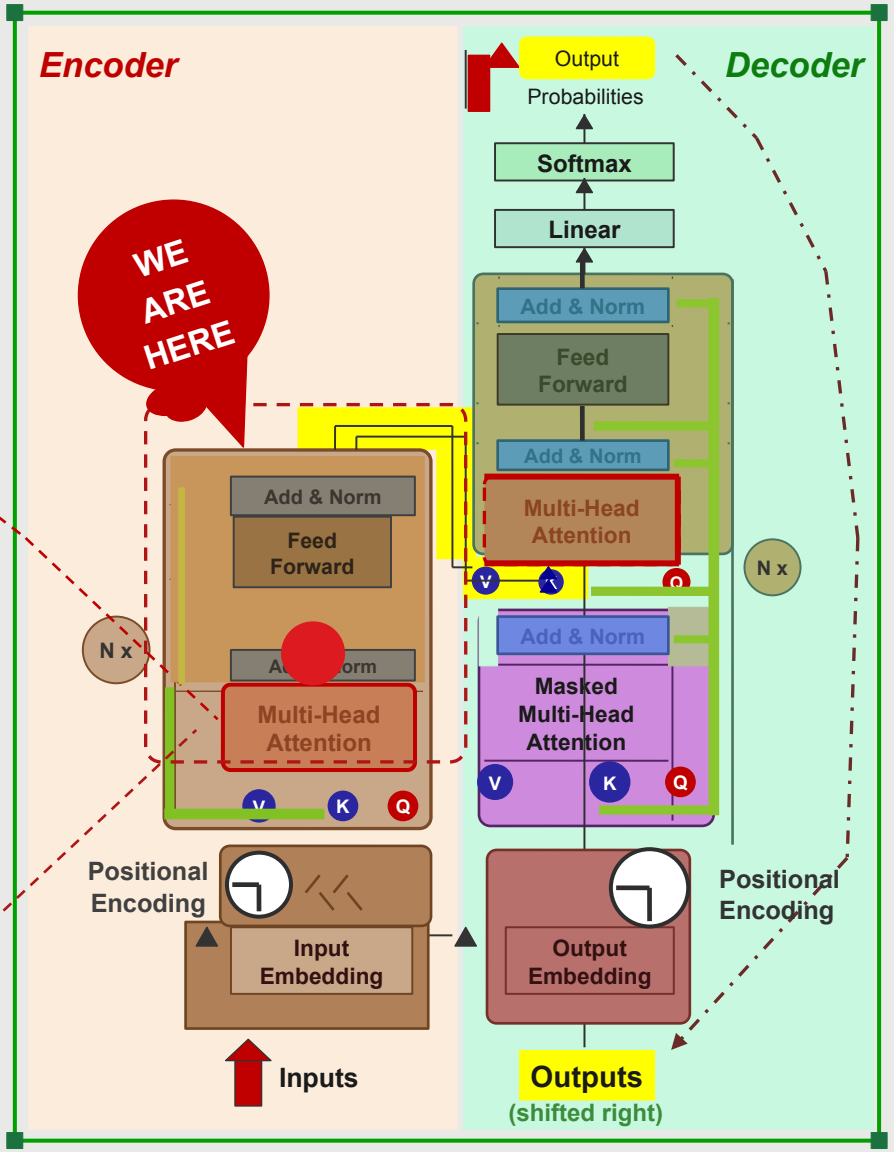
Different Input Sequences should be of Same Length (Fixed Length Sequences)
[Apply Pad or Cut]
Output Sequence is of same Length as Input Sequence



Transformer Blocks (stacked over Multi-Head)







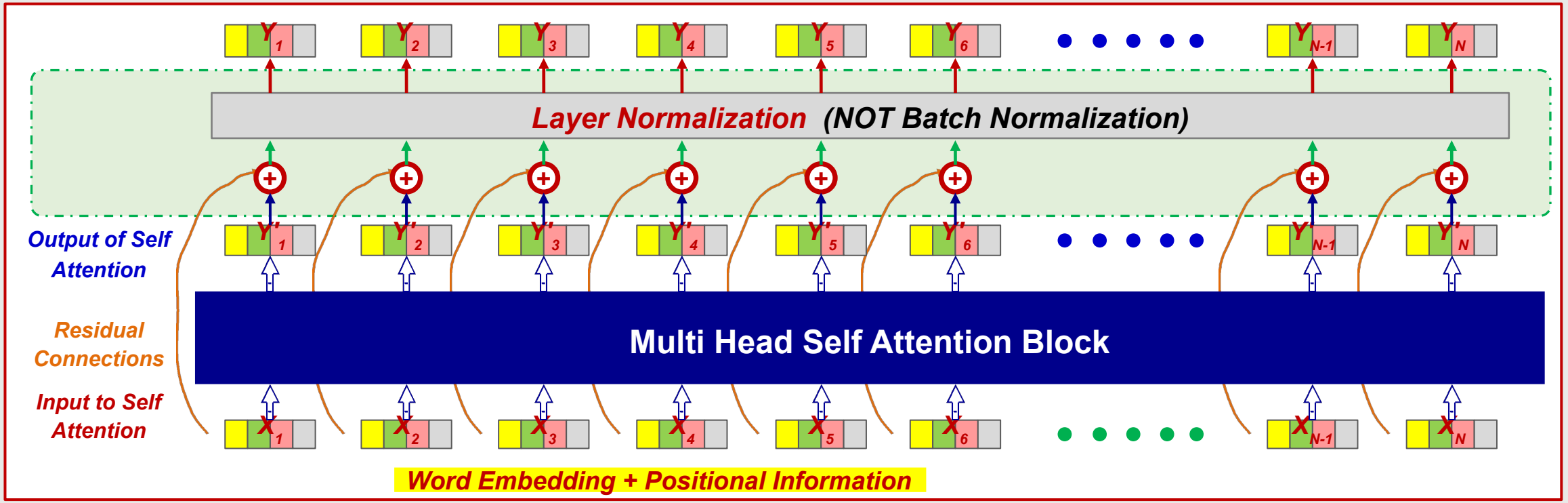
Residual Connections and Normalization

Adding Residual Connection:

Word Embedding + Positional Information [First Transformer Block]
Output of Previous Transformer Block [Stacked Transformer Blocks]

Adding "LAYER" Normalization

Encoded Vectors of Input Sequence [With Same Input Vectors Dimension



Normalization

The behavior of machine learning algorithms changes when the input distribution changes.
{Covariate shift }

layer weights changes during training steps → **Outputs of Activation functions** of each layer **change** → Inputs to the next layer, **(With different Input distribution)**

Conclusion:

Input distribution of each layer changes with each step .

The basic idea behind Normalization is to:

Limit covariate shift by normalizing the activations of each layer

Normalization: Transforming the inputs to be **Zero mean and unit variance.**

Normalization allows each layer to:

- **learn on a more stable distribution of inputs,**
- **Accelerate the training of the network (don't stick with very small learning rates).**

Normalization Parameters

Instead of restricting the Activations of each layer to be *strictly* **Zero** *mean and* **unit** *variance,* Normalization allows the network to **learn parameters** γ and **β** that can *convert the mean and variance to any value that* ***Minimizes Loss.***

Input : Values of x over a mini – batch : $\mathcal{B} = \{x_{1\ldots m}\}$;
Parameters to be learned : $\gamma,\ \beta$

Output : $\{y_i = \mathrm{BN}_{\gamma,\beta}(x_i)\}$

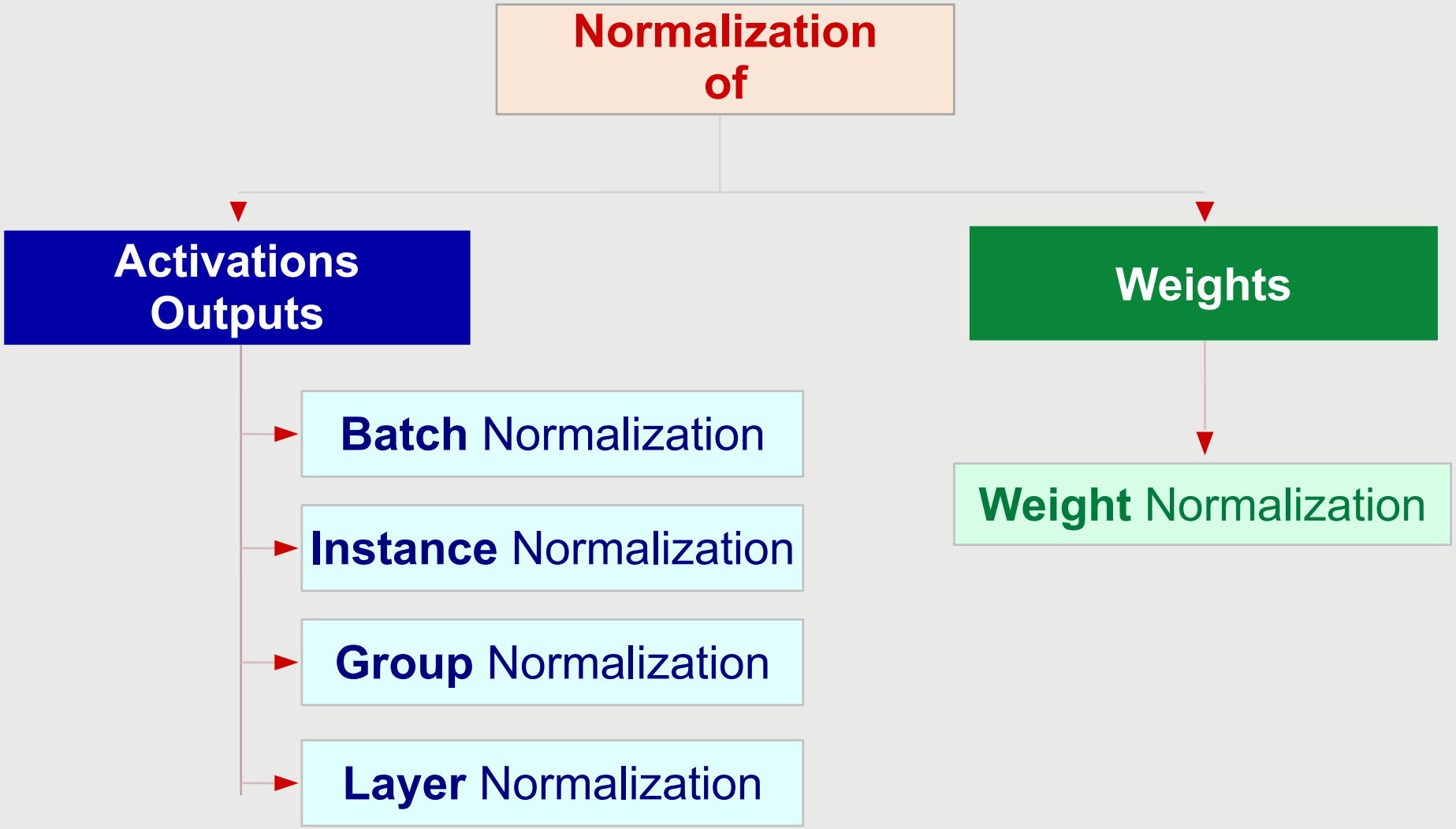
$$\mu_{\mathcal{B}} \leftarrow \frac{1}{m}\sum_{i=1}^{m} x_i \qquad \text{// mini-batch mean}$$

$$\sigma^2_{\mathcal{B}} \leftarrow \frac{1}{m}\sum_{i=1}^{m}(x_i - \mu_{\mathcal{B}})^2 \qquad \text{// mini-batch variance}$$

$$\widehat{x}_i \leftarrow \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma^2_{\mathcal{B}} + \epsilon}} \qquad \text{// normalize}$$

$$y_i \leftarrow \gamma \widehat{x}_i + \beta \equiv \mathrm{BN}_{\gamma,\beta}(x_i) \qquad \text{// scale and shift}$$

Normalization

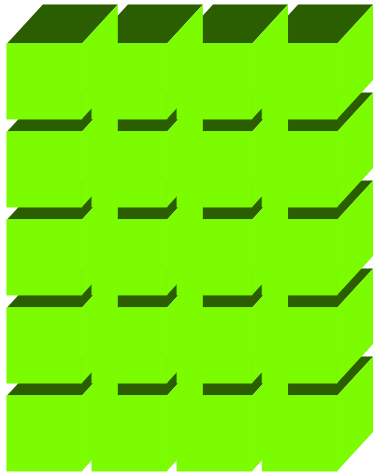


Batch Data (Feature Vectors)

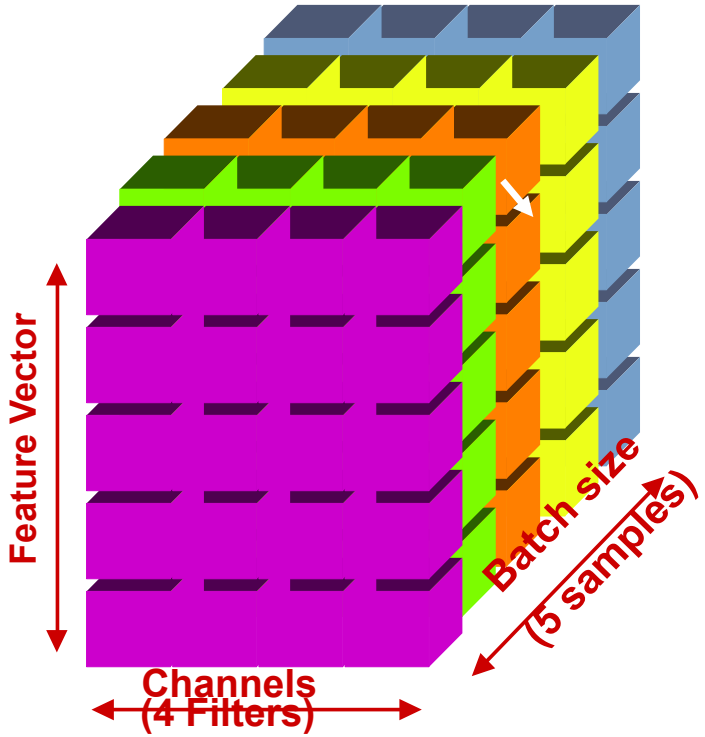
Feature Vector of One Input Sample (Using one filter)



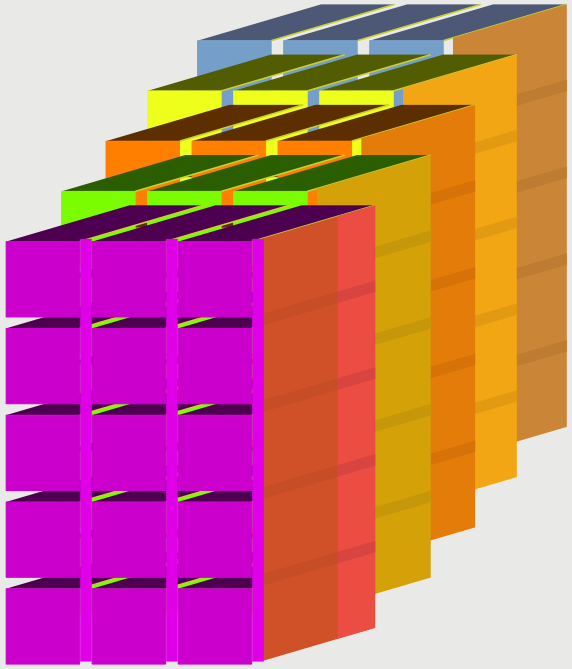
Feature Vectors of One Input Sample (Using 4 filters) [4 Channels]



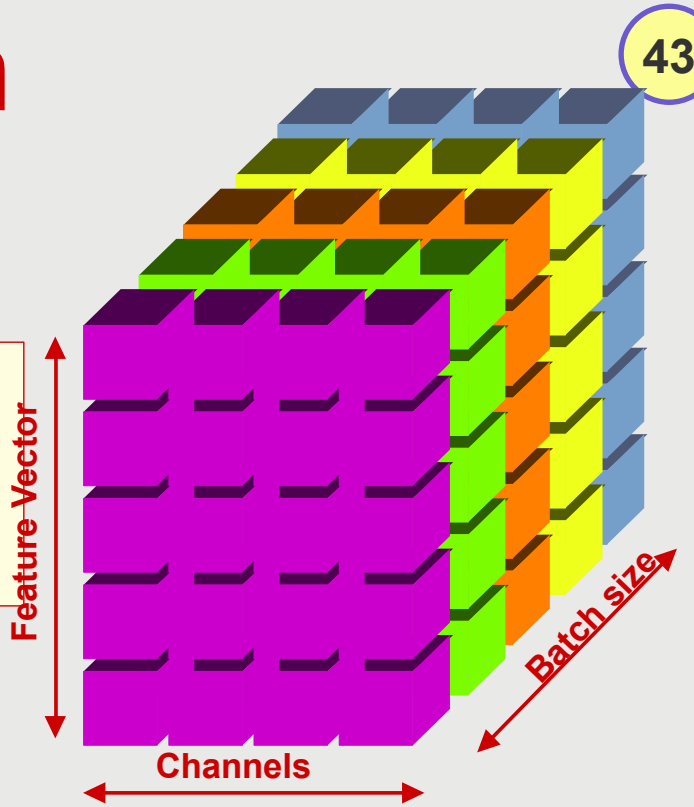
Feature Vectors of :
FIVE Input Samples [mini-Batch]
Using 4 filters [4 Channels]



Batch Normalization



Batch Normalization
Calculate (then Normalize) μ, σ
for **ALL Samples** in the batch
(for each channel)



Batch Normalization Calculate Mean and Variance of Each Mini-Batch instead of Calculating Mean and Variance of the Whole Data.

The mean and variance will differ from mini-batch to another (Source of Error).

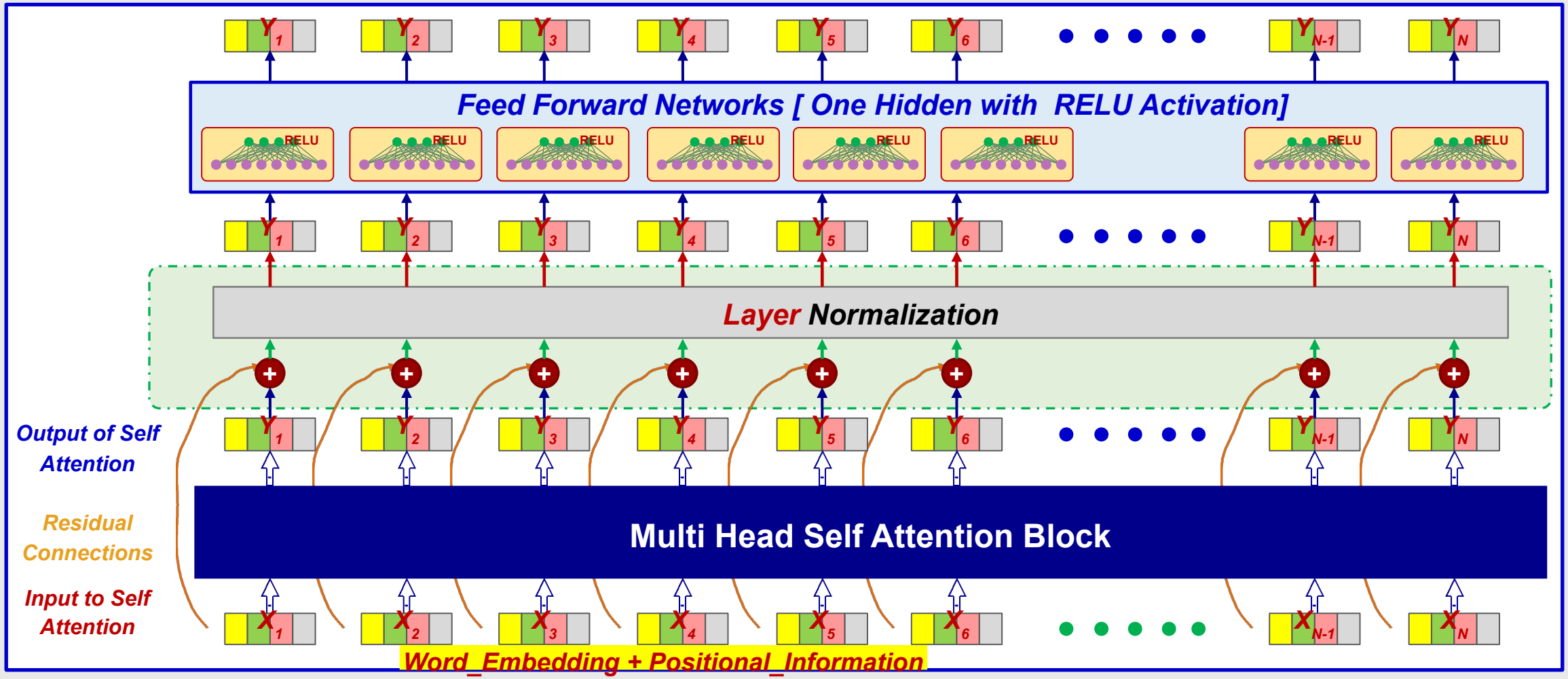
Mini Batch Size should be large enough to minimize "batching" effect. (Constrain)

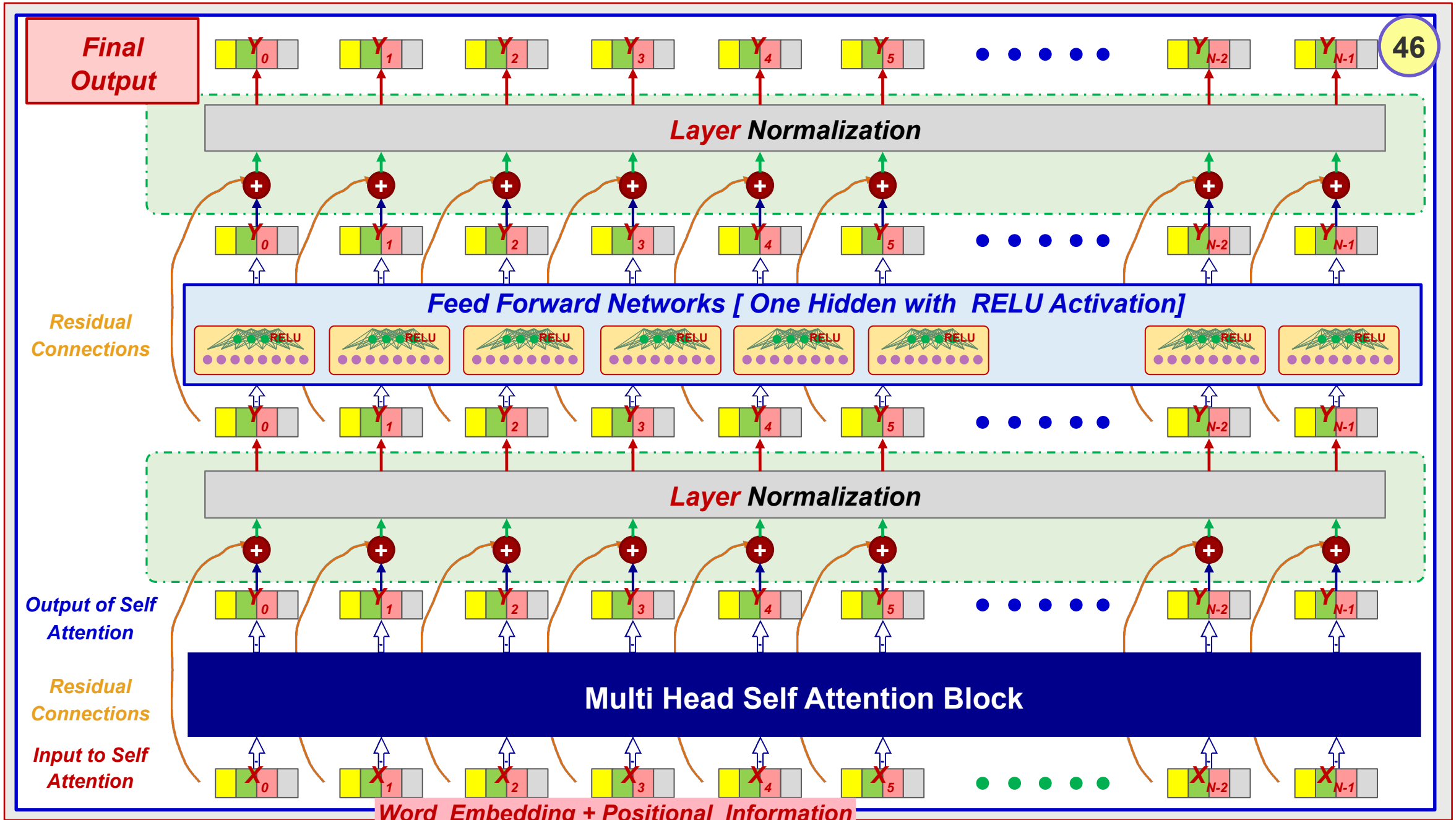
Feed Forward Neural Network

Feed Forward Network for each Vector (After applying Normalization)

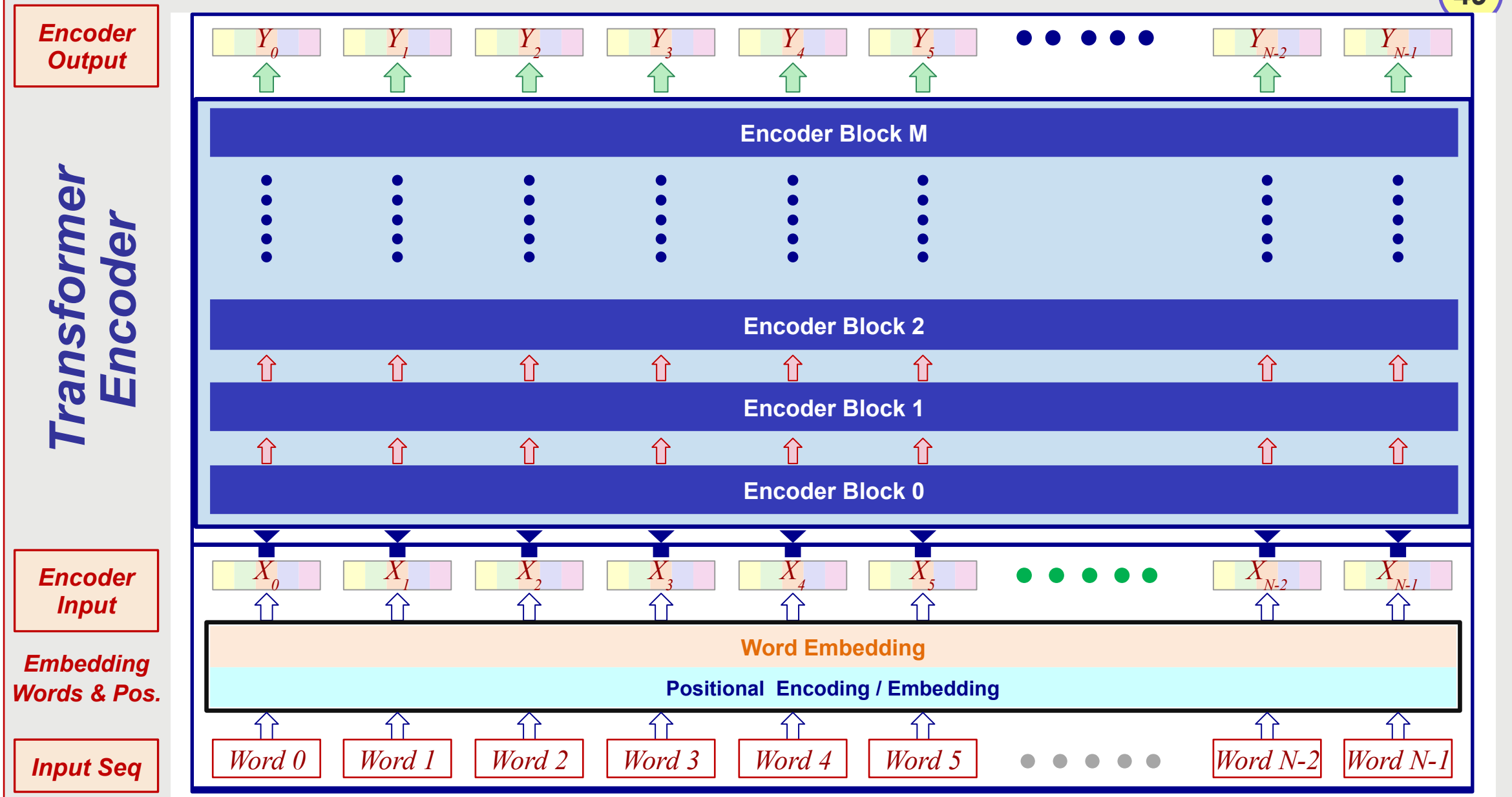
One Hidden Layer with RELU Activation, [Size: at least double Input dimension]

Linear Activation Output Layer [Size: same Input Vector dimension]



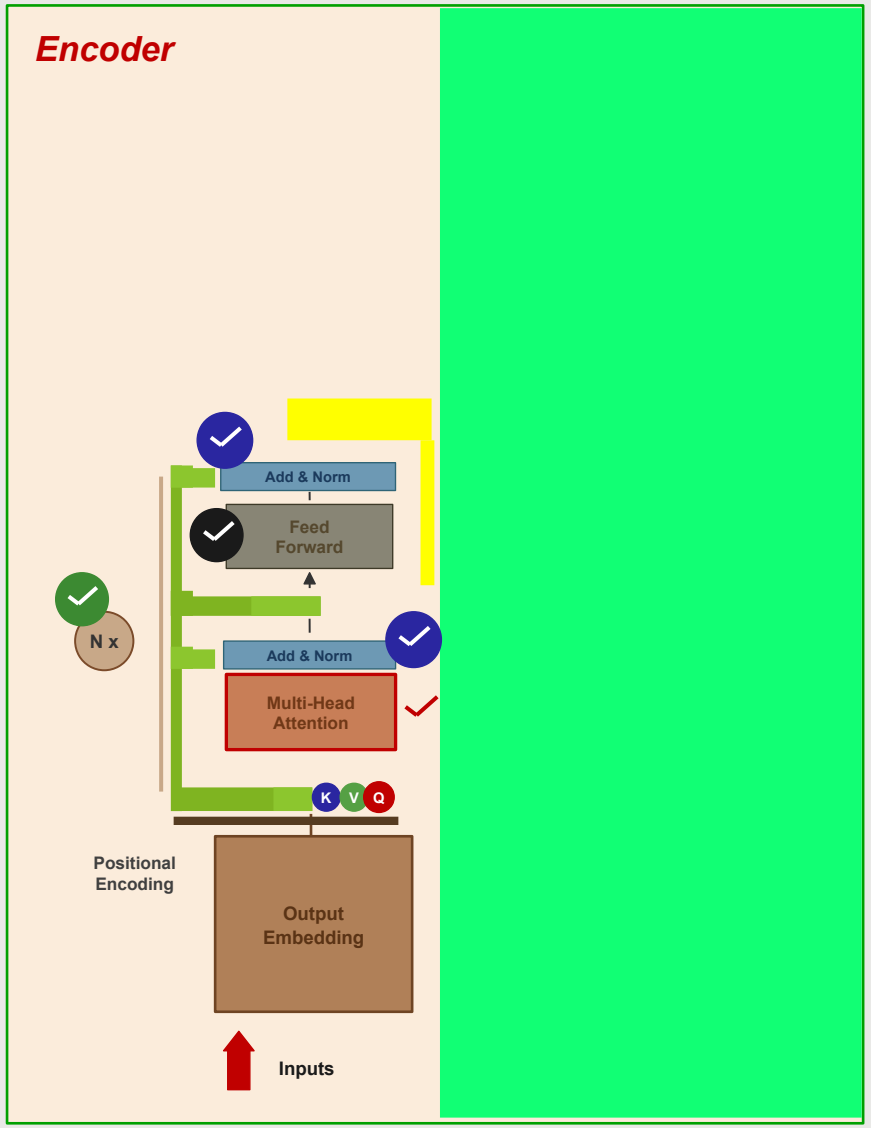

Final Output
46
Layer Normalization
Feed Forward Networks [One Hidden with RELU Activation]
RELU
Residual Connections
Layer Normalization
Output of Self Attention
Residual Connections
Multi Head Self Attention Block
Input to Self Attention
Word Embedding + Positional Information

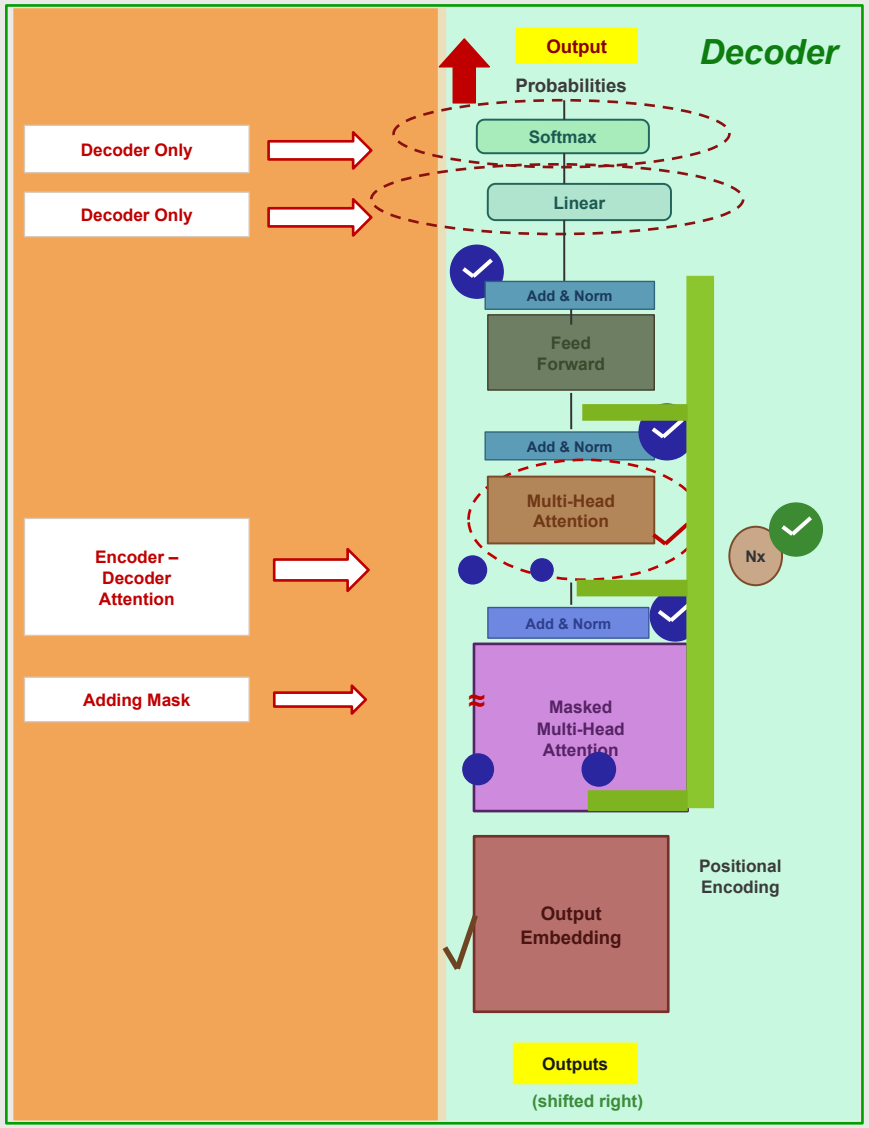
Transformer Stacked Encoder Blocks



Transformer Decoder

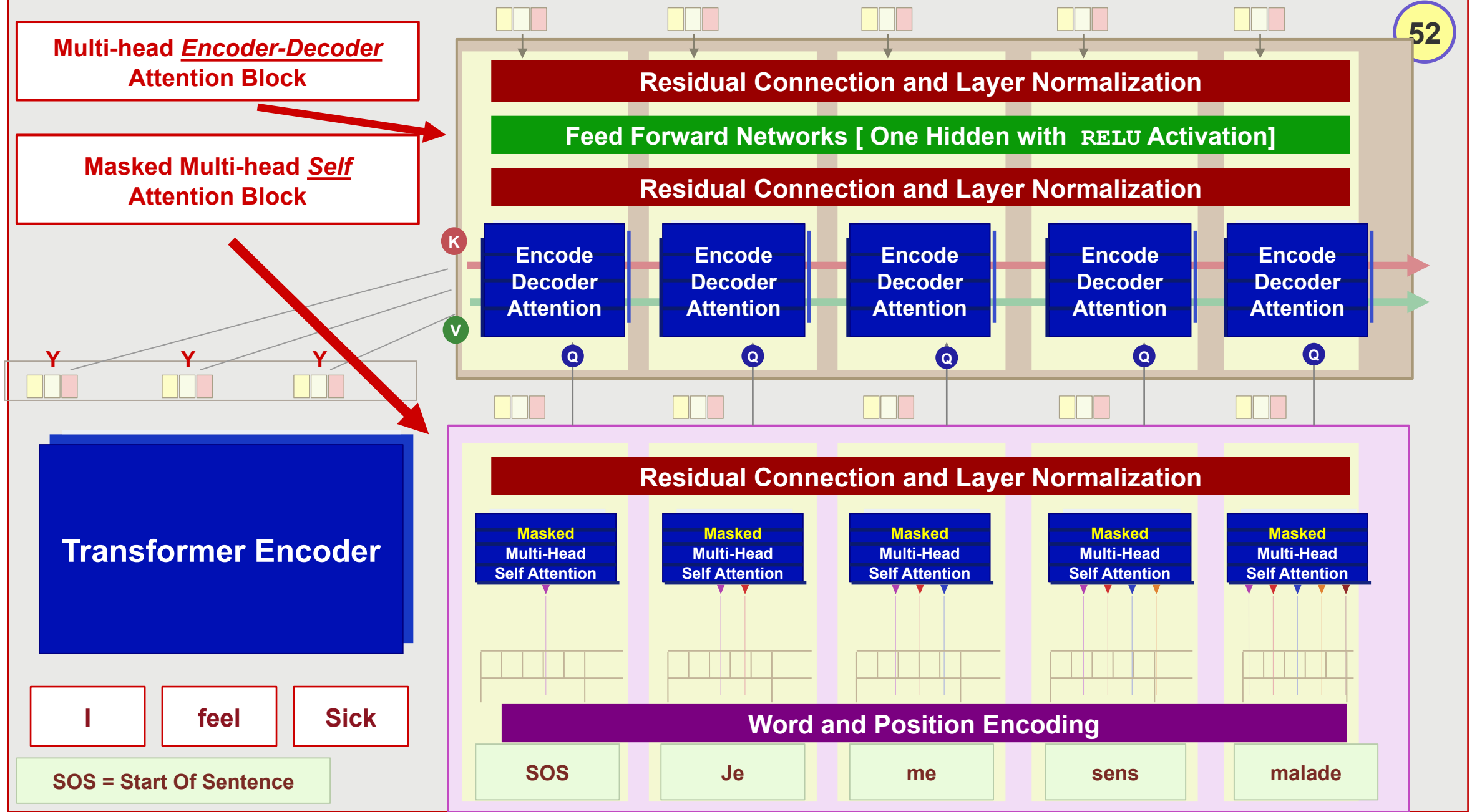
Encoder – Decoder Transformer



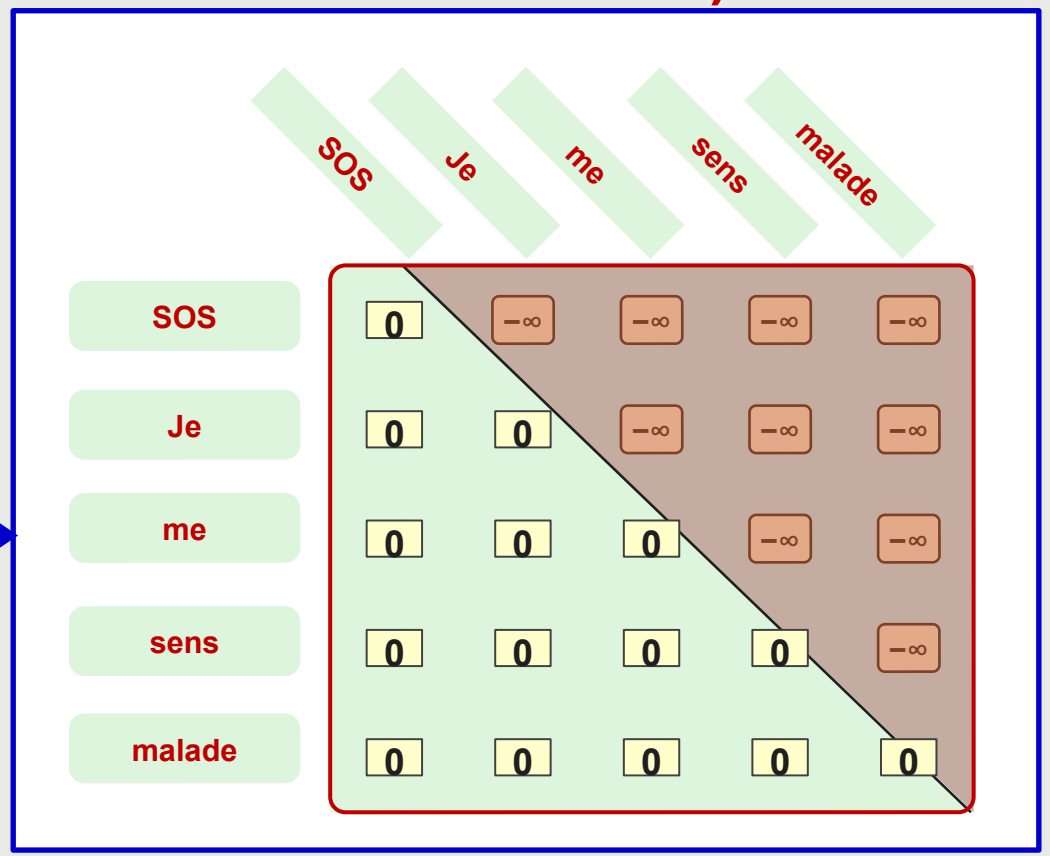


Multi-head *Encoder-Decoder* Attention Block

Masked Multi-head *Self* Attention Block



Self Attention Look Ahead Mask (Used in Decoder first Block)



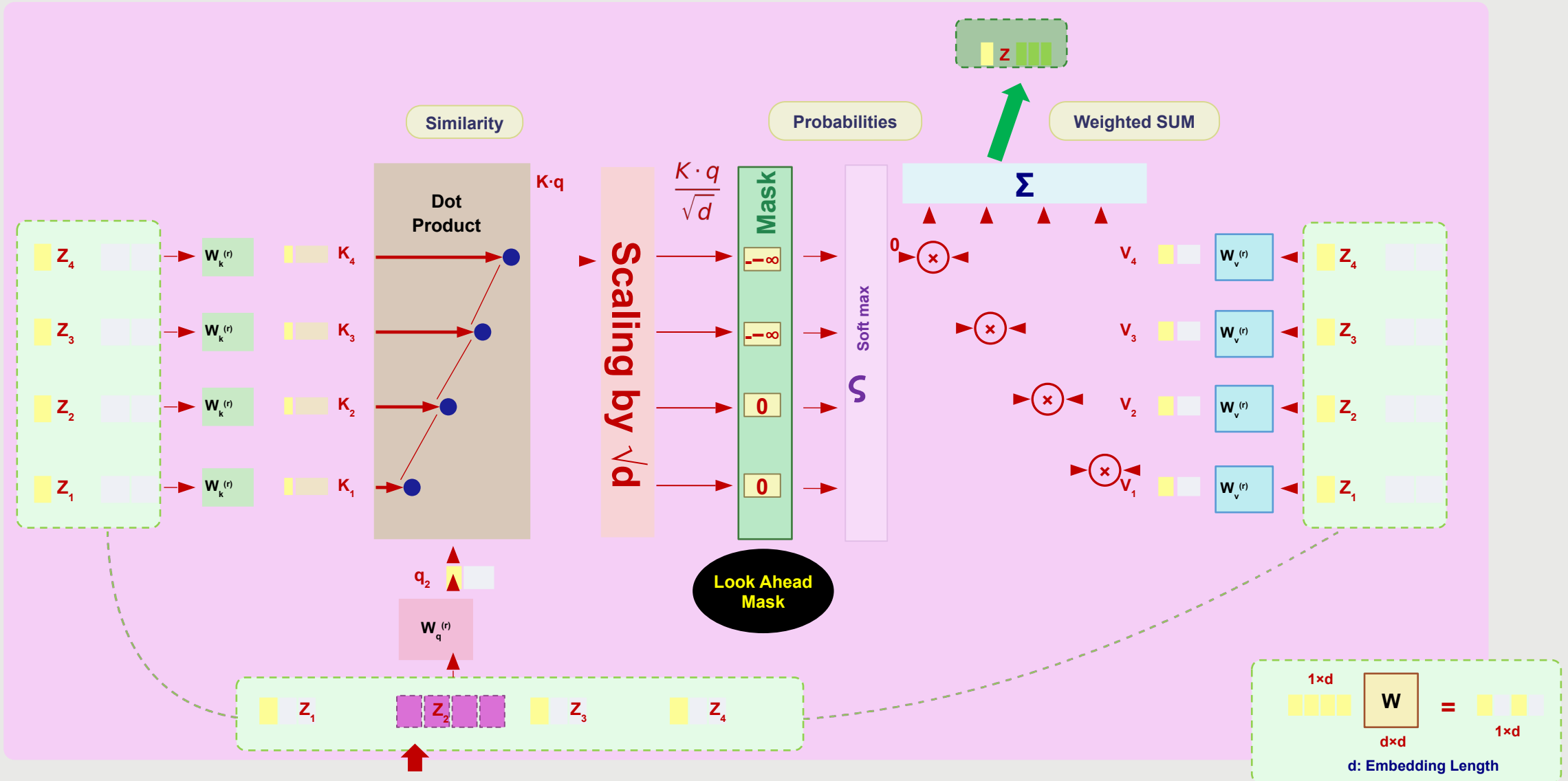
Softmax

$$\sigma(x_j) = \frac{e^{x_j}}{\sum_i e^{x_i}}$$

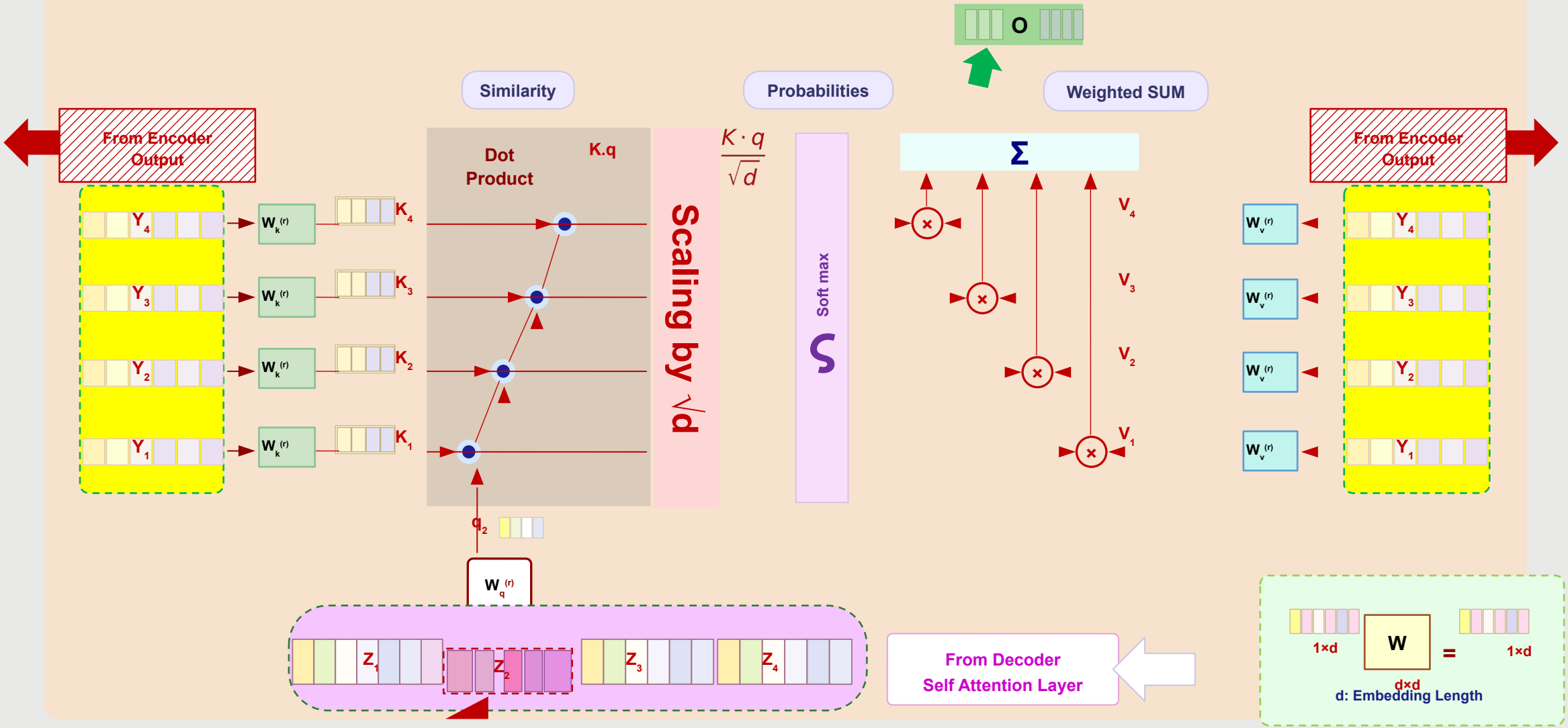
Mask Values are ADDED to Masked (Scaled) Values

Addition of "0" does not change the original values, consequently **no effect on Softmax output**
Addition of "-inf" gets "Zero" output of Softmax, **leaving zero attention scores for future Words.**

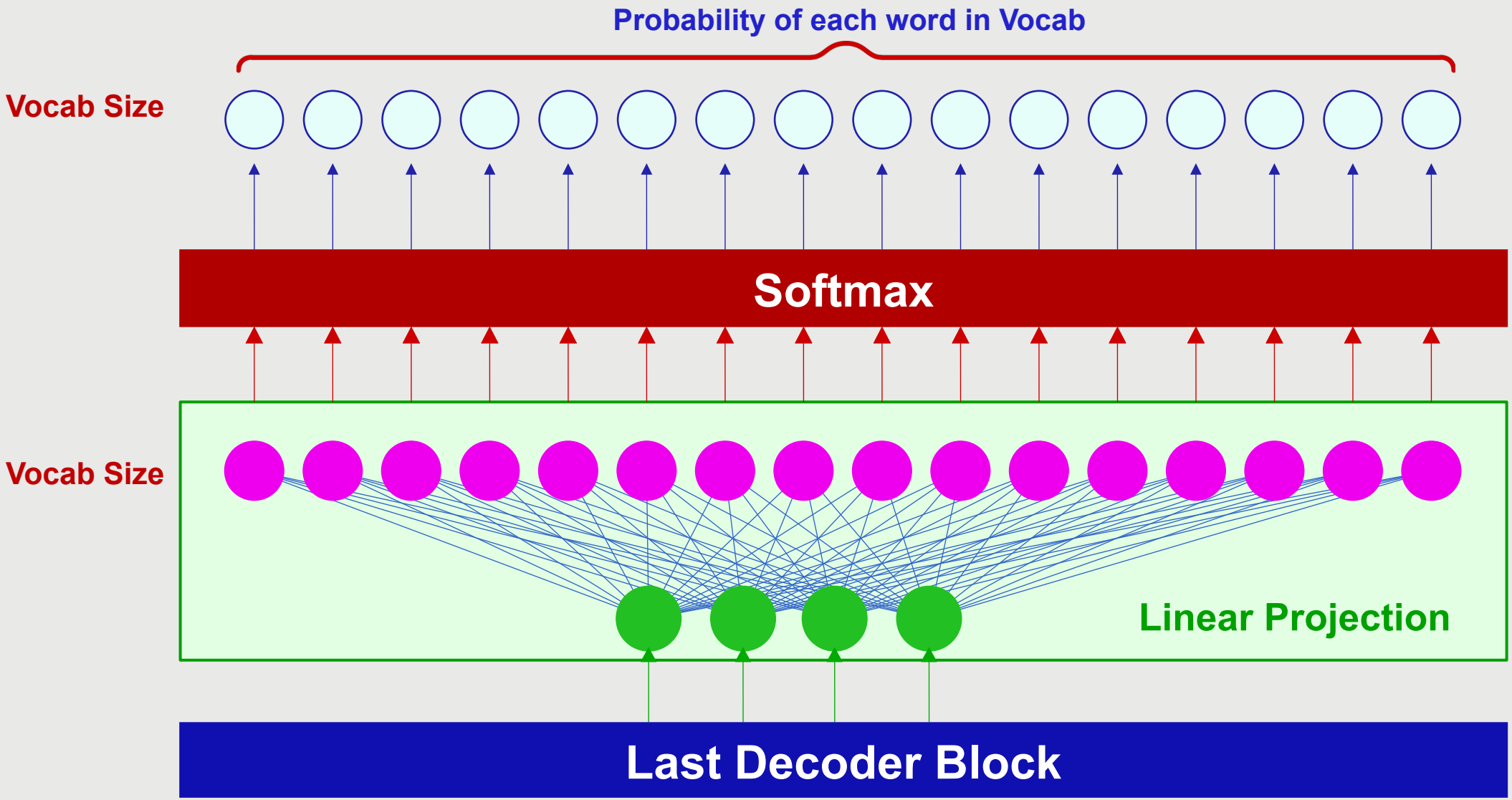
Self Attention Mechanism for Decoder (Adding Mask)



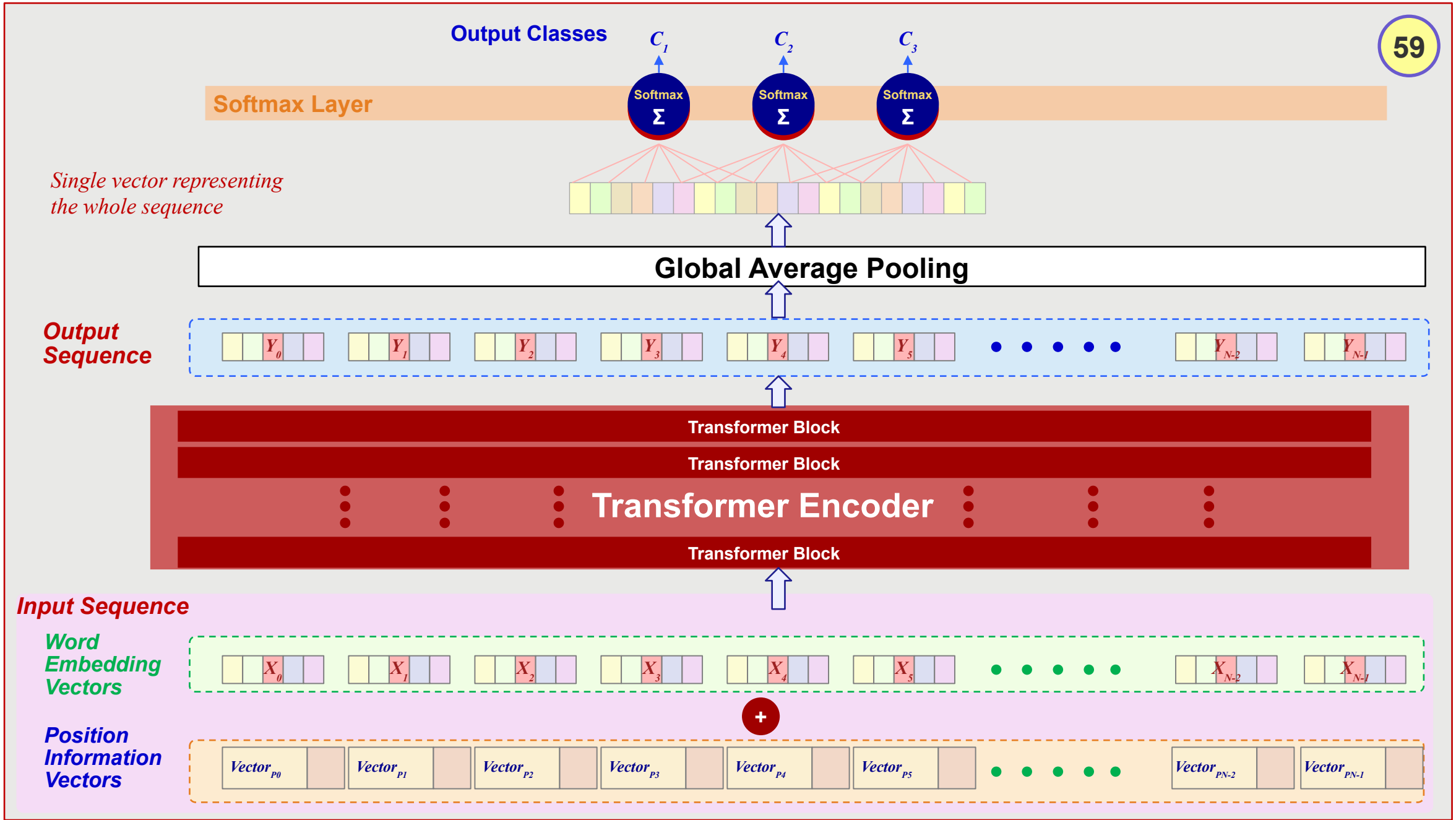
Encoder-Decoder Attention Mechanism (In Decoder)
(Self Attention of word Z_2 w.r.t All words in Sentence (Y_1, Y_2, Y_3, Y_4)



Decoder Final Output



Transformer Encoder as a Classifier


Output Classes
C_1
C_2
C_3
Softmax Layer
Softmax
Softmax
Softmax
Single vector representing the whole sequence
Global Average Pooling
Output Sequence
Y_0 Y_1 Y_2 Y_3 Y_4 Y_5 Y_{N-2} Y_{N-1}
Transformer Block
Transformer Block
Transformer Encoder
Transformer Block
Input Sequence
Word Embedding Vectors
X_0 X_1 X_2 X_3 X_4 X_5 X_{N-2} X_{N-1}
+
Position Information Vectors
$Vector_{P0}$ $Vector_{P1}$ $Vector_{P2}$ $Vector_{P3}$ $Vector_{P4}$ $Vector_{P5}$ $Vector_{PN-2}$ $Vector_{PN-1}$

What is Next (as of 2020)

GPT-1 and GPT-2 (Generative Pre-trained Transformers)

Radford et al., OpenAI. June 2018 and February 2019.

BERT (Bidirectional Encoder Representations from Transformers)

Devlin et al., Google AI Language. May 2019.

RoBERTa (Robustly Optimized BERT Approach)

Liu et al., Facebook AI. June 2019.

Transformer-XL: Attentive Language ModelsBeyond a Fixed-Length Context

Zhang Dai et al. Carnegie Mellon University and Google Brain

T5 (Text-to-Text Transfer Transformer)

GPT-3 (175 Billion Parameter, Beta Release June 2020)